GREENHERB

NOMES:

Guilherme Silva – pv27430

Lucas Pinto – pv27453

Daniel Amaral – pv27449

Rodrigo Santiago - pv25232

1 Sprint 1

O Sprint 1 teve como objetivo implementar a infraestrutura inicial da API GREENHERB em Node.js, desenvolver endpoints de autenticação e criar testes de unidade automatizados para validação do processo de login.

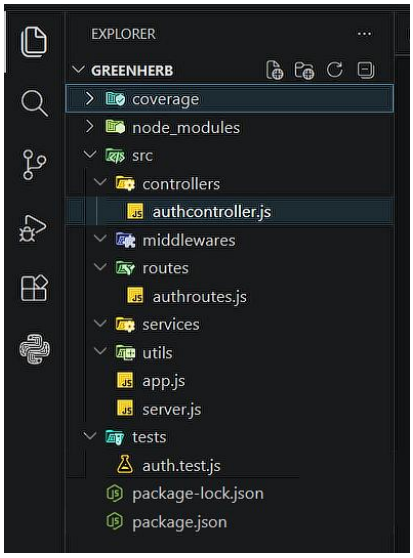
1.1 Tecnologias Utilizadas

Utilizamos as seguintes tecnologias:

Tecnologia	***Objetivo***
Node.js	Backend
Express	API REST
JWT	Autenticação
Jest	Testes Unitários
Supertest	Testes HTTP

1.2 Estrutura do Projeto

O projeto foi organizado em diferentes camadas, separando controladores, rotas e testes, permitindo maior modularidade, organização e facilidade de manutenção.



1.3 Endpoint Implementado

POST /auth/login

O endpoint `/auth/login` foi desenvolvido para permitir a autenticação de utilizadores através de username e password. Quando as credenciais são válidas, a API devolve um token JWT que poderá ser utilizado em endpoints protegidos.

Para validação do endpoint **POST /auth/login**, foi realizado um teste através do PowerShell utilizando o comando `Invoke-RestMethod`, simulando um pedido de autenticação com credenciais válidas.

```
1. Invoke-RestMethod -Uri "http://localhost:3000/auth/login" `
2.   -Method POST `
3.   -ContentType "application/json" `
4.   -Body '{"username":"admin","password":"123456"}'
```

Resultado obtido

O servidor respondeu com sucesso, devolvendo um token JWT:

```
1. {
2. "token": "
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwicm9sZSI6IkFkbWluaXN0cmFkb3IiLCJpYXQiOjE3NzgyMzg3NTIsIm... "
3. }
```

1.4 Estratégia de Testes

Foram desenvolvidos testes de unidade recorrendo a **Jest** e **Supertest**, com o objetivo de validar o comportamento do endpoint de autenticação de forma isolada.

Os testes abrangeram cenários válidos e inválidos, utilizando técnicas de testes de caixa-preta, nomeadamente **particionamento de equivalência** e **análise de valores limite**. O principal objetivo foi garantir que a API responde corretamente a diferentes combinações de entrada e devolve os códigos HTTP apropriados.

1.5 Técnicas de Teste Aplicadas

1.5.1 Particionamento de Equivalência

Esta técnica consistiu em dividir os dados de entrada em classes válidas e inválidas, permitindo reduzir o número de testes necessários sem comprometer a cobertura funcional.

Classe	Tipo	Resultado Esperado
Username Válido	Válida	200
Username inválido	Inválida	401
Password válida	Válida	200
Password inválida	Inválida	401
Campos vazios	Inválida	400

1.5.2 Análise de Valores Limite

A análise de valores limite foi aplicada aos campos obrigatórios do processo de autenticação, de forma a testar comportamentos em condições extremas.

Campo	VaTlor Testado	Resultado Esperado
Password vazia	0 caracteres	Inválido
Password válida mínima	6 caracteres	Válido

1.6 Casos de Teste Implementados

ID	Descrição	Resultado Esperado
TU-01	Login válido	200 + token
TU-02	Username inválido	401

TU-03	Password inválida	401
TU-04	Campos vazios (body {})	400
TU-05	Username nulo	400
TU-06	Password nula	400
TU-07	Username vazio ("")	400
TU-08	Password vazia ("")	400
TU-09	Ausência de username (chave em falta)	400
TU-10	Ausência de password (chave em falta)	400
TU-11	JSON inválido / body mal formado	400
TU-12	Username e password vazios em simultâneo	400
TU-13	Tentativa de injeção no input	400 / 401
TU-14	Rate limiting (tentativas excessivas)	429
TU-15	Comprimento máximo de username excedido	400

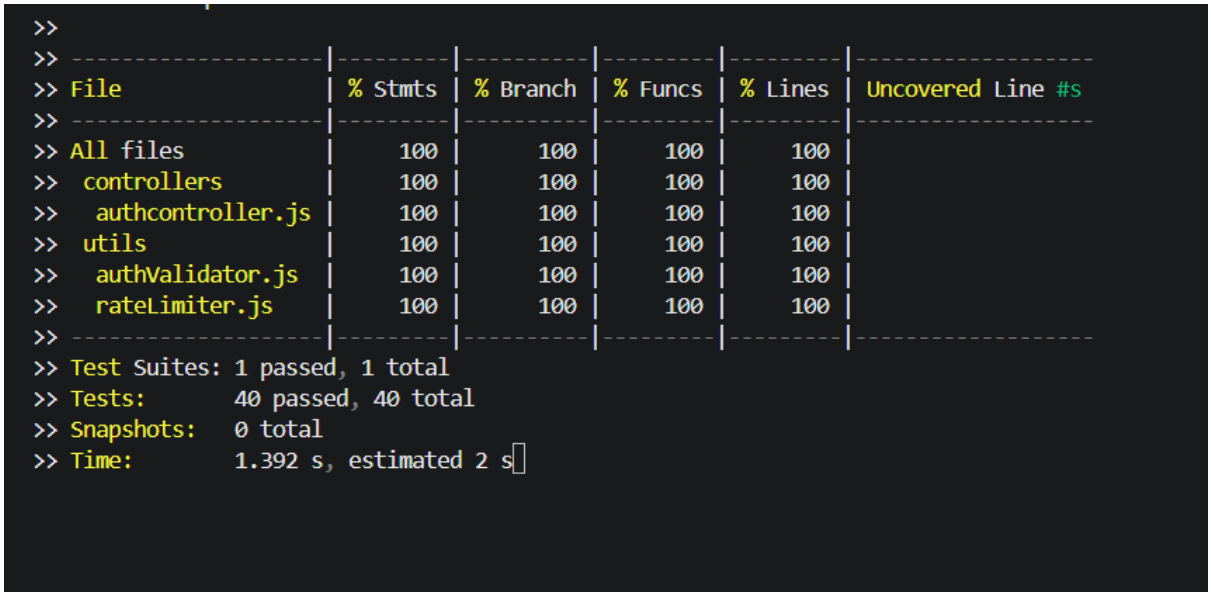
1.7 Resultados da Execução

Todos os testes foram executados com sucesso, validando os diferentes cenários definidos para o endpoint de autenticação.

Foi também obtida **cobertura total de código**, através da ferramenta Jest Coverage.

Métrica	**Resultado**
Statement Coverage	100%
Branch Coverage	100%
Function Coverage	100%
Line Coverage	100%

```
>> PS C:\Users\007lu\Desktop\GreenHerb\GreenHerb> npx jest tests/authUnit.test.js --coverage
>>  PASS  tests/authUnit.test.js
>>   TU-01 — Login válido devolve 200 e token JWT
>>     ✓ credenciais válidas → 200 + token JWT verificável (142 ms)
>>   TU-02 — Username inexistente devolve 401
>>     ✓ utilizador desconhecido → 401 (1 ms)
>>   TU-03 — Password incorreta devolve 401
>>     ✓ utilizador válido mas password errada → 401 (124 ms)
>>   TU-04 — Body completamente vazio devolve 400
>>     ✓ validador isolado com body={} → 400 (1 ms)
>>   TU-05 — username nulo devolve 400
>>     ✓ validador isolado com username=null → 400 username inválido (1 ms)
>>   TU-06 — password nula devolve 400
>>     ✓ validador isolado com password=null → 400 password inválida (1 ms)
>>   TU-07 — username vazio devolve 400/401
>>     ✓ validador com username="" → 400 (2 ms)
>>   TU-08 — password vazia devolve 400/401
>>     ✓ validador com password="" → 400 (1 ms)
>>   TU-09 — body sem chave username devolve 400
>>     ✓ validador com body={password:"..."} → 400 obrigatório (1 ms)
>>   TU-10 — body sem chave password devolve 400
>>     ✓ validador com body={username:"..."} → 400 obrigatório (1 ms)
>>   TU-11 — body não-objeto / mal formado devolve 400
>>     ✓ validador rejeita body do tipo null (1 ms)
>>     ✓ validador rejeita body do tipo undefined (1 ms)
>>     ✓ validador rejeita body do tipo string (1 ms)
>>     ✓ validador rejeita body do tipo número (1 ms)
>>     ✓ validador rejeita body do tipo array (1 ms)
>>   TU-12 — username="" E password="" devolve 400
>>     ✓ ambos vazios → 400 campos obrigatórios inválidos (1 ms)
>>   TU-13 — injeção no username é rejeitada
>>     ✓ payload malicioso ' OR '1'='1 → 400 sem token (2 ms)
>>     ✓ payload malicioso admin'; DROP TABLE users; -- → 400 sem token (1 ms)
>>     ✓ payload malicioso admin' OR 1=1 -- → 400 sem token (1 ms)
>>     ✓ payload malicioso {"$ne": null} → 400 sem token (1 ms)
>>   TU-14 — Rate limiting bloqueia acima de N tentativas
>>     ✓ tentativa N-1 (=4) ainda permite resposta normal (401) (121 ms)
>>     ✓ tentativa N (=5) ainda passa pelo validador (401) (116 ms)
>>     ✓ tentativa N+1 (=6) é bloqueada com 429 (2 ms)
>>   TU-15 — Comprimento máximo do username
>>     ✓ username com limite-1 (=29) chars é aceite pelo validador (1 ms)
```



1.8 Matriz de Rastreabilidade

A matriz de rastreabilidade estabelece a relação entre os requisitos funcionais e os casos de teste implementados.

ID	Requisito / Regra	Endpoint	Nível	Técnica	Resultado Esperado	Pré-condições

TU-01	RF-01: autenticação de utilizador	POST /auth/login	Unidade	Particionamento de Equivalência	200 + token JWT válido	Utilizador válido em mock/stub; sem dependência de BD ou rede
TU-02	RF-01: autenticação – username inválido	POST /auth/login	Unidade	Particionamento de Equivalência	401 credenciais inválidas	Username inexistente passado como parâmetro; sem pré-condições de BD
TU-03	RF-01: autenticação – password inválida	POST /auth/login	Unidade	Particionamento de Equivalência	401 credenciais inválidas	Utilizador existente em mock; password errada passada como parâmetro
TU-04	RN-XX: campos obrigatórios no login	POST /auth/login	Unidade	Valores Limite	400 dados em falta	Body completamente vazio enviado ao validador interno
TU-05	RN-XX: campos obrigatórios – username nulo	POST /auth/login	Unidade	Valores Limite	400 username inválido	username = null passado diretamente ao validador; password

						válida em mock
TU-06	RN-XX: campos obrigatórios – password nula	POST /auth/login	Unidade	Valores Limite	400 password inválida	password = null passado diretamente ao validador; username válido em mock
TU-07	RN-XX: campos obrigatórios – username vazio	POST /auth/login	Unidade	Valores Limite	400/401	username = "" (string vazia); password válida em mock
TU-08	RN-XX: campos obrigatórios – password vazia	POST /auth/login	Unidade	Valores Limite	400/401	password = "" (string vazia); username válido em mock
TU-09	RN-XX: ausência de username	POST /auth/login	Unidade	Particionamento de Equivalência	400 campo obrigatório	Body sem chave username; validador isolado sem BD
TU-10	RN-XX: ausência de password	POST /auth/login	Unidade	Particionamento de Equivalência	400 campo obrigatório	Body sem chave password; validador isolado sem BD

TU-11	RN-XX: validação de formato do request	POST /auth/login	Unidade	Particionamento de Equivalência	400 JSON inválido	Body mal formado (JSON corrompido) enviado ao parser interno
TU-12	RN-XX: combinação de campos vazios	POST /auth/login	Unidade	Condições Múltiplas (username = "", password = "")	400 campos obrigatórios inválidos	username = "" e password = "" em simultâneo; validador isolado
TU-13	RN-XX: segurança – injeção no input	POST /auth/login	Unidade	Particionamento de Equivalência	400/401 sem execução de código	username = ' OR '1'='1; validador de sanitização isolado
TU-14	RN-XX: rate limiting / tentativas excessivas	POST /auth/login	Unidade	Valores Limite (N-1, N, N+1 tentativas)	429 Too Many Requests a partir do limite N	Contador de tentativas em mock; sem BD
TU-15	RN-XX: comprimento máximo de username	POST /auth/login	Unidade	Valores Limite (limite-1, limite, limite+1 chars)	400 input inválido para limite+1	Strings geradas programaticamente no setup do teste

1.9 Conclusão

O Sprint 1 permitiu estabelecer a base da API GREENHERB, incluindo a implementação do sistema de autenticação baseado em JWT e a criação de testes automatizados para validação do login.

A utilização de **Jest** e **Supertest** permitiu validar os diferentes cenários de utilização, garantindo o correto funcionamento do endpoint desenvolvido. Os resultados obtidos demonstram cobertura total de código e comportamento consistente da API perante entradas válidas e inválidas.

2. Sprint 2

O Sprint 2 teve como objetivo implementar testes de unidade para duas funcionalidades centrais da plataforma GREENHERB: a importação do catálogo de ervas aromáticas e a criação de planos de cultivo. De acordo com o plano de sprints definido no enunciado, esta fase incidiu sobre a validação da lógica associada ao catálogo de ervas e aos planos de cultivo, bem como sobre a atualização da matriz de rastreabilidade.

Neste sprint, foram criados dois serviços de domínio: `herbImportService.js` e `planService.js`. Estes serviços permitem isolar a lógica de validação e importação, facilitando a implementação de testes de unidade sem dependência de base de dados, servidor HTTP ou ficheiros externos.

2.1. Tecnologias Utilizadas

Foram utilizadas as mesmas tecnologias já adotadas no Sprint 1, mantendo a coerência da stack de desenvolvimento e testes.

Tecnologia	Objetivo
Node.js	Execução do backend e dos serviços de domínio
Jest	Implementação e execução dos testes unitários
Istanbul/Jest Coverage	Recolha de métricas de cobertura de código
CommonJS	Organização dos módulos através de `require` e `module.exports`

2.2 Estrutura Implementada

No Sprint 2 foram acrescentados novos ficheiros à estrutura do projeto, dedicados à lógica de domínio e aos respetivos testes.

Ficheiro	Objetivo
`src/services/herbImportService.js`	Validação de linhas do catálogo de ervas aromáticas e simulação da importação de dados
`src/services/planService.js`	Validação dos parâmetros necessários para a criação de planos de cultivo
`tests/herbImport.test.js`	Testes de unidade para a importação e validação de ervas aromáticas
`tests/plan.test.js`	Testes de unidade para a validação de planos de cultivo

A separação da lógica em serviços permite testar diretamente as regras de negócio, sem acoplar os testes à camada de rotas ou controladores. Esta abordagem está de acordo com o objetivo dos testes de unidade, que consiste em validar funções e métodos isoladamente, com foco na lógica de domínio.

2.3. Funcionalidades Implementadas

2.3.1 Importação do Catálogo de Ervas Aromáticas

Foi criado o serviço `herbImportService.js`, responsável por validar linhas individuais de ervas aromáticas e simular a importação de um conjunto de linhas.

A função `validateHerbRow(row)` valida os seguintes campos:

Campo	Regra de Validação
`name`	Deve existir, ser do tipo string e não estar vazio
`scientificName`	Deve existir, ser do tipo string e não estar vazio
`minTemperature`	Deve existir e não pode ser negativa
`maxTemperature`	Deve existir e não pode ser negativa
`minTemperature` / `maxTemperature`	A temperatura mínima não pode ser superior à máxima
`minHumidity`	Deve existir e estar entre 0 e 100
`maxHumidity`	Deve existir e estar entre 0 e 100

`minHumidity` / `maxHumidity`	A humidade mínima não pode ser superior à máxima

A função `importHerbs(rows)` recebe um conjunto de linhas e devolve uma resposta agregada com:

Campo	Significado
`imported`	Número de linhas válidas importadas
`invalid`	Número de linhas inválidas
`partial`	Número de linhas inválidas que possuem pelo menos nome ou nome científico
`details`	Lista de erros por linha inválida

2.3.2 Criação de Planos de Cultivo

Foi criado o serviço `planService.js`, responsável por validar os dados necessários para a criação de planos de cultivo.

A função `validatePlan(plan)` valida os seguintes campos:

Campo	Regra de Validação
`type`	Deve ser `regular`, `emergencia` ou `pontual`
`minTemperature` / `maxTemperature`	Devem existir e estar entre 18 ºC e 28 ºC
`minHumidity` / `maxHumidity`	Devem existir e estar entre 40% e 80%
`minLight` / `maxLight`	Devem existir e estar entre 5000 e 25000 lux
`durationDays`	Deve existir e estar entre 1 e 365 dias
`authorizedByResponsible`	Obrigatório para planos do tipo `pontual`

Esta validação está diretamente relacionada com o âmbito definido no enunciado, que destaca a validação de planos de cultivo dos tipos regular, emergência e pontual, incluindo intervalos de temperatura, humidade e luminosidade, bem como a autorização explícita do Responsável Técnico para planos pontuais.

2.4 Estratégia de Testes

Foram desenvolvidos testes de unidade com Jest, incidindo diretamente sobre as funções `validateHerbRow`, `importHerbs` e `validatePlan`.

A estratégia adotada consistiu em testar a lógica de domínio de forma isolada, sem dependência de base de dados, rede, ficheiros CSV/Excel reais ou endpoints HTTP. Esta opção permite uma execução rápida, reprodutível e controlada dos testes.

Foram aplicadas as seguintes técnicas de conceção de casos de teste:

Técnica	Aplicação no Sprint 2
Particionamento de equivalência	Separação de entradas válidas e inválidas para ervas, tipos de plano, campos obrigatórios e ficheiros vazios/inválidos
Análise de valores limite	Teste dos limites de temperatura, humidade, luminosidade e duração dos planos
Condições múltiplas	Validação de decisões compostas, como plano pontual com/sem autorização e verificações com campos obrigatórios combinados

O enunciado exige a aplicação documentada de técnicas formais, nomeadamente particionamento de equivalência, análise de valores limite e cobertura de condições múltiplas, pelo que estes testes foram organizados de forma a justificar explicitamente a seleção dos casos implementados.

2.5 Técnicas de Teste Aplicadas

2.5.1 Particionamento de Equivalência — Catálogo de Ervas Aromáticas

Na importação do catálogo de ervas aromáticas, os dados foram divididos em classes válidas e inválidas, permitindo selecionar casos representativos para cada grupo de entradas.

Parâmetro	Classe Válida	Classe Inválida	Casos de Teste

`name`	String preenchida	Ausente, vazio ou tipo inválido	Linha válida; linha sem nome
`scientificName`	String preenchida	Ausente, vazio ou tipo inválido	Linha válida; linha sem nome científico
`minTemperature`	Valor numérico existente e ≥ 0	Ausente ou negativo	Temperatura mínima em falta
`maxTemperature`	Valor numérico existente e ≥ 0	Ausente ou negativo	Temperatura máxima em falta
Relação entre temperaturas	`minTemperature <= maxTemperature`	`minTemperature > maxTemperature`	Temperatura mínima superior à máxima
`minHumidity` / `maxHumidity`	Valores entre 0 e 100	Valores abaixo de 0 ou acima de 100	Humidade fora do intervalo
Relação entre humidades	`minHumidity <= maxHumidity`	`minHumidity > maxHumidity`	Humidade mínima superior à máxima
`rows`	Array com pelo menos uma linha	Array vazio ou valor não-array	Ficheiro vazio; importação inválida com `null`
Linhas inválidas parciais	Linha inválida com nome ou nome científico	Linha inválida sem identificadores	Linha parcial; linha inválida não parcial

2.5.2 Particionamento de Equivalência — Planos de Cultivo

Na criação de planos de cultivo, o particionamento de equivalência foi aplicado aos tipos de plano, aos campos obrigatórios e às regras específicas dos planos pontuais.

Parâmetro	Classe Válida	Classe Inválida	Casos de Teste
`type`	`regular`, `emergencia`, `pontual`	Tipo ausente ou diferente dos permitidos	Plano regular válido; plano sem tipo; tipo `experimental`
Temperatura	Campos existentes e dentro do intervalo	Campos ausentes, fora do intervalo	Plano sem temperatura; temperatura fora

		ou incoerentes	dos limites; min > max
Humidade	Campos existentes e dentro do intervalo	Campos ausentes, fora do intervalo ou incoerentes	Plano sem humidade; humidade fora dos limites; min > max
Luminosidade	Campos existentes e dentro do intervalo	Campos ausentes, fora do intervalo ou incoerentes	Plano sem luminosidade; luminosidade fora dos limites; min > max
Duração	Campo existente e dentro do intervalo	Campo ausente ou fora do intervalo	Plano sem duração; duração fora dos limites
Autorização de plano pontual	`authorizedByResponsible: true`	`false` ou ausente	Plano pontual autorizado; plano pontual sem autorização

2.5.3 Análise de Valores Limite

A análise de valores limite foi aplicada aos parâmetros numéricos dos planos de cultivo, de acordo com os intervalos definidos para temperatura, humidade, luminosidade e duração. O enunciado recomenda testar o valor imediatamente abaixo do limite inferior, o limite inferior, um valor nominal interior, o limite superior e o valor imediatamente acima do limite superior.

Parâmetro	Intervalo Aceitável	Valores Testados	Resultado Esperado
Temperatura mínima	`[18, 28]`	17, 18, 23, 28, 29	17 e 29 rejeitados; 18, 23 e 28 aceites
Humidade mínima	`[40, 80]`	39, 40, 60, 80, 81	39 e 81 rejeitados; 40, 60 e 80 aceites
Luminosidade mínima	`[5000, 25000]`	4999, 5000, 15000, 25000, 25001	4999 e 25001 rejeitados; 5000, 15000 e 25000 aceites
Duração do ciclo	`[1, 365]`	0, 1, 90, 365, 366	0 e 366 rejeitados; 1, 90 e 365 aceites

Também foram testados valores acima do limite superior para os campos máximos, nomeadamente `maxTemperature = 29`, `maxHumidity = 81` e `maxLight = 25001`, garantindo a validação dos limites superiores dos intervalos.

2.5.4 Cobertura de Condições Múltiplas

A cobertura de condições múltiplas foi aplicada a decisões compostas presentes nos serviços implementados. Em particular, foi analisada a regra de validação do plano pontual:

```
1. if (plan.type === 'pontual' && plan.authorizedByResponsible !== true)
```

Esta decisão é composta por duas condições atómicas:

Condição	Descrição
C1	`plan.type === 'pontual'`
C2	`plan.authorizedByResponsible !== true`

Tabela de verdade:

C1: é pontual?	C2: sem autorização?	Resultado da decisão	Caso de Teste
Verdadeiro	Verdadeiro	Rejeita o plano	Plano pontual sem autorização
Verdadeiro	Falso	Não rejeita por autorização	Plano pontual com autorização
Falso	Verdadeiro	Não rejeita por autorização, pois não é pontual	Plano regular válido
Falso	Falso	Não rejeita por autorização	Plano regular válido / plano não pontual

O conjunto de testes implementado permite verificar que a ausência de autorização só causa rejeição quando o plano é do tipo `pontual`, demonstrando que cada condição influencia o resultado da decisão.

Também foram exercitadas decisões compostas no serviço de importação, nomeadamente:

```
1. if (!Array.isArray(rows) || rows.length === 0)
```

e:

```
1. if (row.name || row.scientificName)
```

Estas condições foram cobertas através de testes com array válido, array vazio, valor `null`, linha inválida parcial e linha inválida não parcial.

2.6 Casos de Teste Implementados

2.6.1 Testes de Importação de Ervas Aromáticas

ID	Descrição	Resultado Esperado
TU-16	Linha válida deve ser aceite	Resultado `valid: true` e sem erros
TU-17	Linha sem nome deve ser rejeitada	Erro “Nome obrigatório”
TU-18	Linha com temperatura mínima superior à máxima deve ser rejeitada	Erro de incoerência entre temperaturas
TU-19	Linha com humidade fora do intervalo deve ser rejeitada	Erros de humidade mínima e máxima inválidas
TU-20	Importação com linhas válidas e inválidas deve devolver totais corretos	1 importada, 2 inválidas, 2 parciais
TU-21	Importação de ficheiro vazio deve devolver erro	Erro “Ficheiro vazio ou inválido”
TU-22	Linha sem nome científico deve ser rejeitada	Erro “Nome científico obrigatório”
TU-23	Linha com temperatura mínima em falta deve ser rejeitada	Erro “Temperatura mínima inválida”
TU-24	Linha com temperatura máxima em falta deve ser rejeitada	Erro “Temperatura máxima inválida”
TU-25	Linha com humidade mínima superior à máxima deve ser rejeitada	Erro de incoerência entre humidades
TU-26	Importação inválida quando os dados não são array	Erro “Ficheiro vazio ou inválido”
TU-27	Linha inválida sem nome e sem nome científico não deve contar como parcial	0 importadas, 1 inválida, 0 parciais

2.6.2 Testes de Criação de Planos de Cultivo

ID	Descrição	Resultado Esperado
TU-28	Plano regular válido deve ser aceite	Resultado `valid: true` e sem erros
TU-29	Tipo de plano inválido deve ser rejeitado	Erro “Tipo de plano inválido”
TU-30	Valores limite da temperatura mínima	17 e 29 rejeitados; 18, 23 e 28 aceites
TU-31	Valores limite da humidade mínima	39 e 81 rejeitados; 40, 60 e 80 aceites
TU-32	Valores limite da luminosidade mínima	4999 e 25001 rejeitados; 5000, 15000 e 25000 aceites
TU-33	Valores limite da duração do plano	0 e 366 rejeitados; 1, 90 e 365 aceites
TU-34	Plano pontual sem autorização deve ser rejeitado	Erro de autorização do Responsável Técnico
TU-35	Plano pontual com autorização e parâmetros válidos deve ser aceite	Resultado `valid: true` e sem erros
TU-36	Temperatura mínima superior à máxima deve ser rejeitada	Erro de incoerência entre temperaturas
TU-37	Plano sem temperatura deve ser rejeitado	Erro “Temperatura obrigatória”
TU-38	Plano sem humidade deve ser rejeitado	Erro “Humidade obrigatória”
TU-39	Plano sem luminosidade deve ser rejeitado	Erro “Luminosidade obrigatória”
TU-40	Plano sem duração deve ser rejeitado	Erro “Duração obrigatória”
TU-41	Plano sem tipo deve ser rejeitado	Erro “Tipo de plano inválido”
TU-42	Plano com temperatura máxima fora do intervalo deve ser rejeitado	Erro “Temperatura máxima fora do intervalo permitido”
TU-43	Plano com humidade máxima fora do intervalo deve ser rejeitado	Erro “Humidade máxima fora do intervalo permitido”
TU-44	Plano com luminosidade máxima fora do intervalo deve ser rejeitado	Erro “Luminosidade máxima fora do intervalo permitido”

TU-45	Plano com humidade mínima superior à máxima deve ser rejeitado	Erro de incoerência entre humidades
TU-46	Plano com luminosidade mínima superior à máxima deve ser rejeitado	Erro de incoerência entre luminosidades

Nota: os testes parametrizados com `test.each` originam múltiplas execuções individuais no Jest. Assim, embora estejam representados aqui como casos agregados por regra de validação, a execução automática contabiliza cada valor testado como teste individual, contribuindo para o total de 51 testes executados.

2.7 Resultados da Execução

A bateria de testes foi executada com sucesso através do comando:

```
1. npm test
```

Resultado obtido:

```
Test Suites: 3 passed, 3 total
Tests:       51 passed, 51 total
Snapshots:   0 total
Time:        0.57 s
```

Posteriormente, foi executado o relatório de cobertura através do comando:

```
1. npm run coverage
```

A execução confirmou cobertura total em todos os ficheiros analisados:

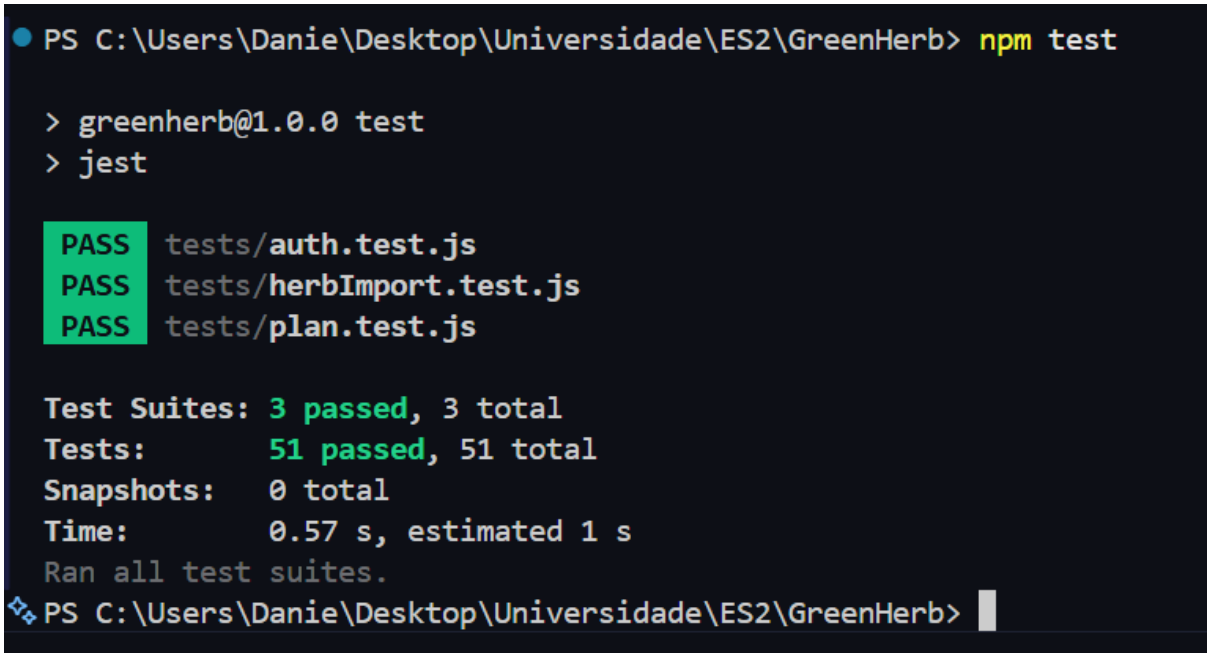
Métrica	Resultado
Statement Coverage	100%
Branch Coverage	100%
Function Coverage	100%
Line Coverage	100%

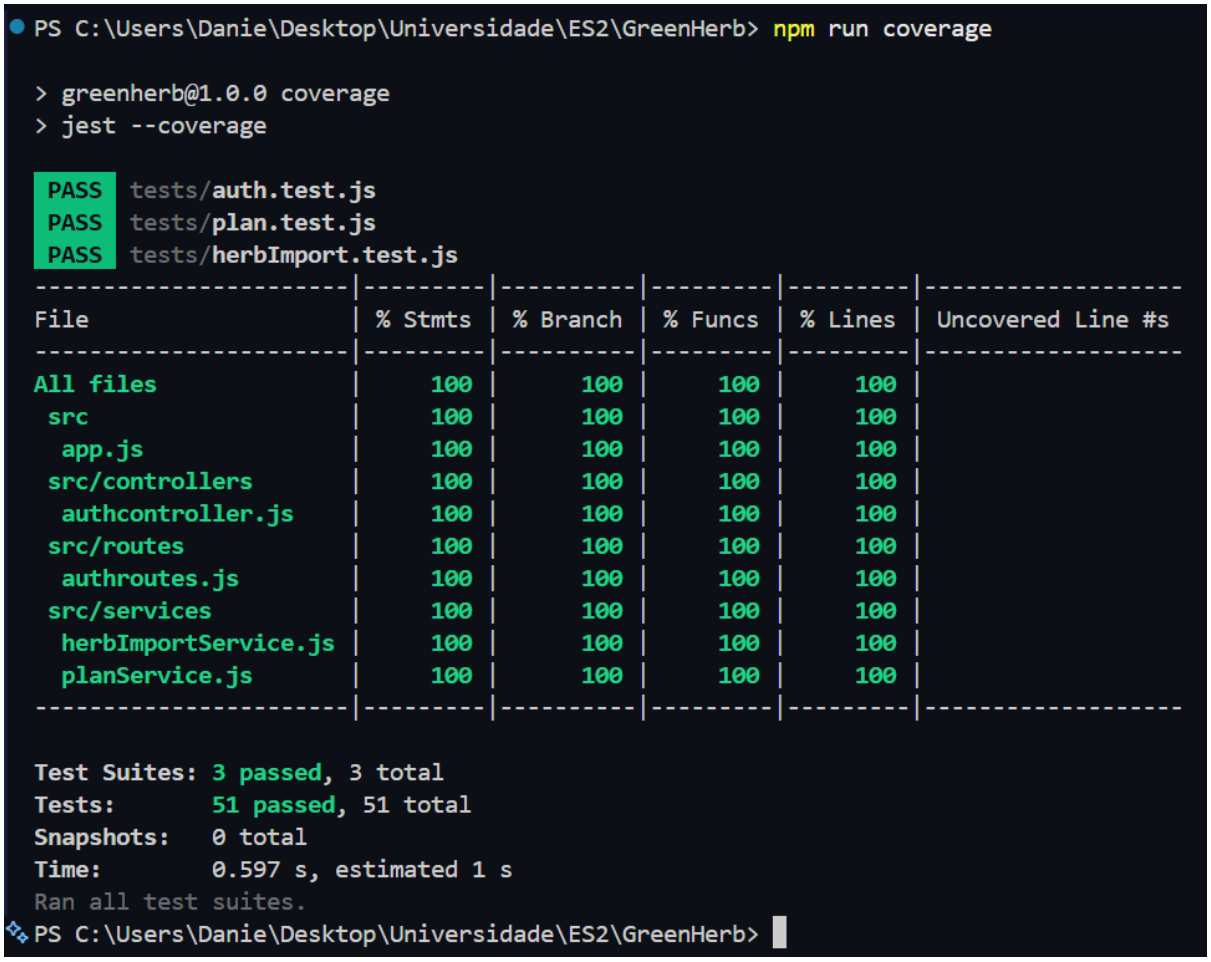
Cobertura por ficheiro:

Ficheiro	Statements	Branches	Functions	Lines

app.js	100%	100%	100%	100%
authcontroller.js	100%	100%	100%	100%
authroutes.js	100%	100%	100%	100%
herbImportService.js	100%	100%	100%	100%
planService.js	100%	100%	100%	100%

Imagens das execuções:





Estes resultados demonstram que todos os caminhos de execução, ramos condicionais, funções e linhas dos ficheiros produtivos foram exercitados pela bateria de testes implementada.

2.8 Matriz de Rastreabilidade

A matriz de rastreabilidade estabelece a relação entre os requisitos funcionais, as regras de negócio e os casos de teste implementados. O enunciado define que cada linha da matriz deve conter, no mínimo, o ID do caso de teste, requisito/regra de negócio, endpoint ou componente exercitado, nível de teste, técnica aplicada, resultado esperado e pré-condições .

2.8.1 Matriz Sprint 2 — Caso de Teste → Requisito

ID	Requisito / Regra	Endpoint / Componente	Nível	Técnica	Resultado Esperado	Pré-condições
TU-16	RF3-HERB-01: validar linha válida do catálogo de ervas	`validateHerbRow()`	Unidade	Particionamento de Equivalência	Linha aceite, `valid: true`	Objeto de erva completo e válido
TU-17	RF-HERB-02: nome da erva obrigatório	`validateHerbRow()`	Unidade	Particionamento de Equivalência	Linha rejeitada com “Nome obrigatório”	Objeto de erva com `name` vazio
TU-18	RN-HERB-01: temperatura mínima não pode	`validateHerbRow()`	Unidade	Particionamento de Equivalência / regra de coerência	Linha rejeitada com erro de temperatura	Objeto com `minTemperature > maxTemperature`

	exceder máxima					
TU-19	RN-HERB-02: humidade deve estar entre 0 e 100	`validateHerbRow()`	Unidade	Valores Limite / Particionamento de Equivalência	Linha rejeitada por humidade inválida	Objeto com `minHumidity < 0` e `maxHumidity > 100`
TU-20	RF-HERB-03: importação com linhas válidas, inválidas e parciais	`importHerbs()`	Unidade	Particionamento de Equivalência	Totais agregados corretos	Array com uma linha válida e duas inválidas
TU-21	RF-HERB-04: rejeição de importação vazia	`importHerbs()`	Unidade	Particionamento de Equivalência	Erro “Ficheiro vazio ou inválido”	Array vazio
TU-22	RF-HERB-05: nome científico obrigatório	`validateHerbRow()`	Unidade	Particionamento de Equivalência	Linha rejeitada com “Nome científico obrigatório”	Objeto com `scientificName` vazio
TU-	RF-HERB-06: tempera	`validateHerbRow()`	Unidade	Particionamento de Equivalência	Linha rejeitada com “Tempera	Objeto sem `minTemperature`

23	tura mínima obrigatória				tura mínima inválida”	
TU-24	RF-HERB-07: temperatura máxima obrigatória	`validateHerbRow()`	Unidade	Particionamento de Equivalência	Linha rejeitada com “Temperatura máxima inválida”	Objeto sem `maxTemperature`
TU-25	RN-HERB-03: humidade mínima não pode exceder máxima	`validateHerbRow()`	Unidade	Particionamento de Equivalência / regra de coerência	Linha rejeitada com erro de humidade	Objeto com `minHumidity > maxHumidity`
TU-26	RF-HERB-08: dados de importação devem ser array	`importHerbs()`	Unidade	Particionamento de Equivalência / condições múltiplas	Erro “Ficheiro vazio ou inválido”	Valor `null` passado à função
TU-27	RN-HERB-04: linha inválida só conta como parcial se tiver	`importHerbs()`	Unidade	Condições múltiplas	Linha inválida não incrementa `partial`	Linha sem `name` e sem `scientificName`

	identificador					
TU-28	RF-PLAN-01: plano regular válido	`validatePlan()`	Unidade	Particionamento de Equivalência	Plano aceite, `valid: true`	Objeto de plano regular válido
TU-29	RF-PLAN-02: tipo de plano inválido	`validatePlan()`	Unidade	Particionamento de Equivalência	Plano rejeitado com “Tipo de plano inválido”	Plano com `type: "experimental"`
TU-30	RN-PLAN-01: temperatura deve estar entre 18 e 28 ºC	`validatePlan()`	Unidade	Valores Limite	17 e 29 rejeitados; 18, 23 e 28 aceites	Plano regular com temperatura mínima variável
TU-31	RN-PLAN-02: humidade deve estar entre 40 e 80%	`validatePlan()`	Unidade	Valores Limite	39 e 81 rejeitados; 40, 60 e 80 aceites	Plano regular com humidade mínima variável
TU-32	RN-PLAN-03: luminosidade deve estar entre 5000 e	`validatePlan()`	Unidade	Valores Limite	4999 e 25001 rejeitados; 5000, 15000 e 25000 aceites	Plano regular com luminosidade mínima variável

	25000 lux					
TU-33	RN-PLAN-04: duração deve estar entre 1 e 365 dias	`validatePlan()`	Unidade	Valores Limite	0 e 366 rejeitados; 1, 90 e 365 aceites	Plano regular com duração variável
TU-34	RN-PLAN-05: plano pontual exige autorização	`validatePlan()`	Unidade	Condições Múltiplas	Plano rejeitado por falta de autorização	Plano pontual com `authorizedByResponsible: false`
TU-35	RN-PLAN-06: plano pontual autorizado pode ser aceite	`validatePlan()`	Unidade	Condições Múltiplas	Plano aceite quando autorizado e válido	Plano pontual com `authorizedByResponsible: true`
TU-36	RN-PLAN-07: temperatura mínima não pode exceder máxima	`validatePlan()`	Unidade	Particionamento de Equivalência / regra de coerência	Plano rejeitado por incoerência de temperatura	Plano com `minTemperature > maxTemperature`
TU-	RF-PLAN-03:	`validatePlan()`	Unidade	Particionamento de	Plano rejeitado com	Plano sem `minTemperature` e `maxTemperature`

37	temperatura obrigatória			Equivalência	“Temperatura obrigatória”	
TU-38	RF-PLAN-04: humidade obrigatória	`validatePlan()`	Unidade	Particionamento de Equivalência	Plano rejeitado com “Humidade obrigatória”	Plano sem `minHumidity` e `maxHumidity`
TU-39	RF-PLAN-05: luminosidade obrigatória	`validatePlan()`	Unidade	Particionamento de Equivalência	Plano rejeitado com “Luminosidade obrigatória”	Plano sem `minLight` e `maxLight`
TU-40	RF-PLAN-06: duração obrigatória	`validatePlan()`	Unidade	Particionamento de Equivalência	Plano rejeitado com “Duração obrigatória”	Plano sem `durationDays`
TU-41	RF-PLAN-07: tipo de plano obrigatório	`validatePlan()`	Unidade	Particionamento de Equivalência	Plano rejeitado com “Tipo de plano inválido”	Plano sem `type`
TU-42	RN-PLAN-08: temperatura máxima deve respeitar limite superior	`validatePlan()`	Unidade	Valores Limite	Plano rejeitado com erro de temperatura máxima	Plano com `maxTemperature = 29`

TU-43	RN-PLAN-09: humidade máxima deve respeitar limite superior	`validatePlan()`	Unidade	Valores Limite	Plano rejeitado com erro de humidade máxima	Plano com `maxHumidity = 81`
TU-44	RN-PLAN-10: luminosidade máxima deve respeitar limite superior	`validatePlan()`	Unidade	Valores Limite	Plano rejeitado com erro de luminosidade máxima	Plano com `maxLight = 25001`
TU-45	RN-PLAN-11: humidade mínima não pode exceder máxima	`validatePlan()`	Unidade	Particionamento de Equivalência / regra de coerência	Plano rejeitado por incoerência de humidade	Plano com `minHumidity > maxHumidity`
TU-46	RN-PLAN-12: luminosidade mínima não pode exceder máxima	`validatePlan()`	Unidade	Particionamento de Equivalência / regra de coerência	Plano rejeitado por incoerência de luminosidade	Plano com `minLight > maxLight`

2.8.2 Cobertura Bidirecional — Requisito → Casos de Teste

Requisito / Regra	Casos de Teste Associados
RF-HERB-01: validar linha válida do catálogo de ervas	TU-16
RF-HERB-02: nome da erva obrigatório	TU-17
RN-HERB-01: temperatura mínima não pode exceder máxima	TU-18
RN-HERB-02: humidade deve estar entre 0 e 100	TU-19
RF-HERB-03: importação com linhas válidas, inválidas e parciais	TU-20
RF-HERB-04: rejeição de importação vazia	TU-21
RF-HERB-05: nome científico obrigatório	TU-22
RF-HERB-06: temperatura mínima obrigatória	TU-23
RF-HERB-07: temperatura máxima obrigatória	TU-24
RN-HERB-03: humidade mínima não pode exceder máxima	TU-25
RF-HERB-08: dados de importação devem ser array	TU-26
RN-HERB-04: linha inválida parcial exige identificador	TU-27
RF-PLAN-01: plano regular válido	TU-28
RF-PLAN-02: tipo de plano inválido	TU-29
RN-PLAN-01: temperatura deve estar entre 18 e 28 ºC	TU-30
RN-PLAN-02: humidade deve estar entre 40 e 80%	TU-31
RN-PLAN-03: luminosidade deve estar entre 5000 e 25000 lux	TU-32
RN-PLAN-04: duração deve estar entre 1 e 365 dias	TU-33
RN-PLAN-05: plano pontual exige autorização	TU-34
RN-PLAN-06: plano pontual autorizado pode ser aceite	TU-35
RN-PLAN-07: temperatura mínima não pode exceder máxima	TU-36
RF-PLAN-03: temperatura obrigatória	TU-37
RF-PLAN-04: humidade obrigatória	TU-38
RF-PLAN-05: luminosidade obrigatória	TU-39
RF-PLAN-06: duração obrigatória	TU-40
RF-PLAN-07: tipo de plano obrigatório	TU-41
RN-PLAN-08: temperatura máxima deve respeitar limite superior	TU-42

RN-PLAN-09: humidade máxima deve respeitar limite superior	TU-43
RN-PLAN-10: luminosidade máxima deve respeitar limite superior	TU-44
RN-PLAN-11: humidade mínima não pode exceder máxima	TU-45
RN-PLAN-12: luminosidade mínima não pode exceder máxima	TU-46

Esta tabela demonstra a rastreabilidade bidirecional dos testes do Sprint 2, permitindo identificar tanto o requisito coberto por cada caso de teste como os casos de teste associados a cada requisito. Esta abordagem responde à exigência do enunciado de que a matriz evidencie a inexistência de requisitos sem cobertura .

2.9 Defeitos Detetados

Durante a implementação e execução dos testes do Sprint 2, foi identificado um comportamento inicialmente ambíguo na contagem de linhas parciais durante a importação de ervas aromáticas.

ID	Descrição	Severidade	Passos para Reproduzir	Estado
DEF-01	A contagem de linhas parciais podia ser interpretada de forma ambígua, uma vez que qualquer linha inválida com `name` ou `scientificName` era classificada como parcial	Baixa	Executar importação com linhas inválidas contendo apenas um identificador textual	Resolvido/Clarificado nos testes
DEF-02	A cobertura de ramos não atingia inicialmente 100% devido à ausência de teste para linha inválida sem `name` e sem `scientificName`	Baixa	Executar `npm run coverage` antes da inclusão do teste adicional	Resolvido

2.10 Conclusão

O Sprint 2 permitiu reforçar significativamente a cobertura de testes da plataforma GREENHERB, acrescentando testes de unidade sobre a lógica de importação do catálogo de ervas aromáticas e sobre a criação de planos de cultivo.

Foram aplicadas técnicas de particionamento de equivalência, análise de valores limite e cobertura de condições múltiplas, garantindo que os casos de teste não foram escolhidos de forma arbitrária, mas sim de acordo com critérios formais de conceção. Esta preocupação é particularmente relevante, uma vez que o enunciado valoriza o rigor metodológico e a justificação das opções tomadas, e não apenas a quantidade de testes executados .

A execução final da bateria de testes apresentou 51 testes aprovados e cobertura total de 100% em instruções, ramos, funções e linhas. Desta forma, o Sprint 2 cumpre os objetivos definidos, deixando a base preparada para a implementação futura de testes de integração e sistema.

3. Sprint 3

O Sprint 3 teve como objetivo principal a implementação de testes de unidade para todos os requisitos funcionais e regras de negócio que não tinham sido abrangidos nos sprints anteriores. Este sprint foi fundamental para garantir que a lógica de negócio crítica da plataforma está devidamente testada e validada, recorrendo a técnicas formais de teste de software.

Os objetivos específicos deste sprint incluíram:

- Aplicar técnicas de teste de caixa-preta, como o particionamento de equivalência e a análise de valores limite, para validar o comportamento do sistema face a entradas válidas e inválidas.
- Utilizar técnicas de teste de caixa-branca, nomeadamente cobertura de condições múltiplas (MCC) e cobertura MC/DC, para garantir que todas as decisões lógicas no código foram testadas de forma exaustiva.
- Atualizar a matriz de rastreabilidade, de modo a garantir que cada caso de teste está associado a um requisito funcional ou regra de negócio específica, e que todos os requisitos críticos estão cobertos.
- Alcançar uma cobertura de 100% da lógica de negócio não testada anteriormente, com especial foco em áreas como transições de estado de lotes, classificação de alertas, regras de automação, registo de auditoria e validação de perfis de utilizador.

3.2 Estrutura Implementada

No Sprint 3 foram adicionados novos ficheiros à estrutura do projeto, dedicados à implementação da lógica de domínio dos requisitos restantes e aos respetivos testes de unidade.

Tipo	Ficheiro	Descição
Testes de Unidade	`src/services/batchService.js`	Implementação das transições de estado de lotes e cálculo de produtividade.
	`src/services/alertService.js`	Lógica de classificação de alertas e validação de justificações.
	`src/services/automationService.js`	Gestão da comutação entre modos Manual/Automático e

		decisão de execução/sugestão de ações.
	`src/services/auditService.js`	Registo de ações críticas para auditoria.
	`src/services/userService.js`	Validação de perfis e permissões de utilizadores.
Testes de Unidade	`tests/batchService.test.js`	Testes para transições de estado e cálculo de produtividade.
	`tests/alertService.test.js`	Testes para classificação de alertas e validação de justificações.
	`tests/automationService.test.js`	Testes para comutação de modos e decisão de ações.
	`tests/auditService.test.js`	Testes para registo de auditoria.
	`tests/userService.test.js`	Testes para validação de perfis de utilizadores.

3.3 Funcionalidades Implementadas

3.3.1 Transições de Estado de Lotes

Foi criado o serviço *`batchService.js`*, responsável por gerir as transições de estado dos lotes de cultivo (`ativo`, `concluído`, `comprometido`) e o cálculo de produtividade com base em perdas e divisões.

A função *`transitionBatch(batch, user, newState, justification)`* valida as seguintes regras:

Campo	Regras de Validação
`batch.state`	Deve ser `ativo` para permitir transições.
`newState`	Deve ser `concluído` ou `comprometido`.
`batch.losses`	Se `newState = 'concluído'` e `losses > 0`, é obrigatória uma **justificação**.
`user.role`	Apenas `Responsável Técnico` ou `Administrador` podem aprovar transições.

`batch.losses`	Se `newState = 'comprometido'`, é obrigatório que `losses > 0` e uma **justificação**.

A função devolve o lote atualizado ou lança um erro se as regras não forem cumpridas.

3.3.2 Classificação e Resolução de Alertas

Foi criado o serviço *`alertService.js`*, responsável por classificar alertas com base em medições ambientais e limites definidos, bem como validar a resolução ou ignorar alertas com justificação.

A função *`classifyAlert(temperature, humidity, sensorOK, limits)`* valida as seguintes regras:

Condições	Classificações
`!sensorOK`	
`` `temperatura > maxTemp ``	
Todos os valores dentro dos limites	

A função *`resolveAlert(alert, action, justification)`* valida as seguintes regras:

Ação	Requisitos	Resultado
`resolve`	Justificação opcional.	`status: 'resolved'`
`invalid`	Ação não permitida.	Lança erro: `"Ação inválida"`
`ignore`	Justificação obrigatória (10-500 caracteres).	`status: 'ignored'`

3.3.3 Comutação de Modos de Automação

Foi criado o serviço `automationService.js`, responsável por gerir a comutação entre os modos Manual e Automático, bem como decidir se uma ação deve ser executada ou sugerida.

A função *switchMode(currentMode, newMode, user)* valida as seguintes regras:

Condição	Resultado
newMode é válido (Manual ou Automático)	Devolve newMode.
newMode é inválido	Lança erro: "Modo inválido. Use 'Manual' ou 'Automático'."
newMode = 'Automático' **e** user.role é técnico	Lança erro: "Não autorizado: Apenas Responsável Técnico ou Administrador pode comutar para modo Automático."

A função *decideAction(mode, ruleActive, measurement)* decide se uma ação deve ser executada ou sugerida:

Modo	Regra Ativa	Resultado
Automático	true	execute
Automático	false	suggest
Manual	true ou false	suggest

3.3.4 Registo de Auditoria

Foi criado o serviço *auditService.js*, responsável por registar todas as ações críticas do sistema, incluindo o utilizador, a ação e o *timestamp*.

A função *logAction(user, action)* valida as seguintes regras:

Condição	Resultado
user ou action em falta	Lança erro: "Utilizador e ação são obrigatórios para registo de auditoria."
user e action válidos	Regista a entrada no array auditLogs e devolve o objeto de log.

A função *isAuditable(action)* verifica se uma ação deve ser registada:

Ação	Auditável?
criou_plano	Sim
transicionou_lote	Sim
resolveu_alerta	Sim
GET, fetch, read	Não

3.3.5 Validação dos Perfis dos Utilizadores

Foi criado o serviço *`userService.js`*, responsável por validar os perfis de utilizador e as suas permissões para realizar determinadas ações.

A função *`validateUserRole(user, requiredRoles)`* valida as seguintes regras:

Condição	Resultado
`user.role` está em `requiredRoles`	Devolve `true`.
`user.role` não está em `requiredRoles`	Lança erro: `"Não autorizado: Perfil '${user.role}' não tem permissão para esta ação."`
`user.role` não está em `requiredRoles`	Lança erro: `"Utilizador é obrigatório e deve ter um perfil."`

A função *`getUserPermissions(role)`* devolve as permissões associadas a cada perfil:

Perfil	Pode Aprovar Planos	Pode Trocar Modo	Pode Transacionar Lotes
`Técnico`	Não	Não	Não
`Responsável Técnico`	Sim	Sim	Sim
`Administrador`	Sim	Sim	Sim

3.4 Estratégia de Teste

Foram desenvolvidos testes de unidade com Jest, incidindo diretamente sobre as funções implementadas nos serviços descritos. A estratégia adotada consistiu em testar a lógica de domínio de forma isolada, sem dependência de base de dados, rede ou *endpoints* HTTP. Esta abordagem permite uma execução rápida, reprodutível e controlada dos testes.

Foram aplicadas as seguintes técnicas de conceção de casos de teste:

Técnia	Aplicação no Sprint
Particionamento de Equivalência	Separação de entradas válidas e inválidas para transições de estado, classificação de alertas, e perfis de utilizador.
Análise de Valores Limite	Teste dos limites de justificação (9, 10, 500, 501 caracteres) e valores numéricos (e.g., `losses = 0`).

Cobertura de Condições Múltiplas	Validação de decisões compostas, como transições de estado com perdas e justificação.
MC/DC Coverage	Teste de cada condição atómica em decisões compostas (e.g., `classifyAlert`).

O enunciado exige a aplicação documentada de técnicas formais, nomeadamente particionamento de equivalência, análise de valores limite e cobertura de condições múltiplas, pelo que os testes foram organizados de forma a justificar explicitamente a seleção dos casos implementados

3.5 Técnicas de Teste Aplicadas

3.5.1 Particionamento de Equivalência

Na validação das transições de estado de lotes, os dados foram divididos em classes válidas e inválidas:

Parâmetro	Classe Válida	Classe Invalida	Casos de Teste
`user.role`	`'Responsável Técnico'`, `'Administrador'`	`'Técnico'`	Transição com utilizador autorizado.
`newState`	`'concluído'`, `'comprometido'`	`'ativo'`, `'invalid'`	Transição para `concluído` ou `comprometido`.
`batch.state`	`'ativo'`	`'concluído'`, `'comprometido'`	Transição de `ativo` para `concluído`.
`batch.losses`	`0`	`> 0` sem justificação	Transição com `losses = 0` ou `losses = 5`.

Na classificação de alertas, os dados foram divididos em classes válidas e inválidas:

Parâmetro	Classe Válida	Classe Invalida	Casos de Teste
`temperature`	Dentro dos limites (`[18, 28]`)	Fora dos limites (`< 18` ou `> 28`)	`temperature = 20`, `temperature = 30`.
`humidity`	Dentro dos limites (`[40, 80]`)	Fora dos limites (`< 40` ou `> 80`)	`humidity = 50`, `humidity = 85`.
`sensorOK`	`true`	`false`	`sensorOK = true`, `sensorOK = false`.

3.5.2 Análise de Valores Limite

A análise de valores limite foi aplicada à justificação para ignorar alertas:

Parâmetro	Intervalo Aceitável	Valores Testados	Resultado Esperado
`justification.length`	[10, 500]	9, 10, 250, 500, 501	9 e 501 rejeitados; 10, 250, 500 aceites.

3.5.3 Cobertura de Condições Múltplas

A cobertura de condições múltiplas foi aplicada à função `transitionBatch`, que depende de múltiplas condições:

C1: estadoAtual = 'ativo'	C2: losses > 0	C3: justificação válida	Resultado	Caso de Teste
Verdadeiro	Verdadeiro	Verdadeiro	Transição permitida	`ativo → concluído` com justificação.
Verdadeiro	Verdadeiro	Verdadeiro	Lança erro	`ativo → concluído` sem justificação.
Verdadeiro	Verdadeiro		Transição permitida	`ativo → concluído` sem justificação.
Verdadeiro	-	-	Lança	`concluído → ativo`.

3.5.4 Matriz de Rastreabilidade

ID	Requisito / Regra	Componente	Nível	Técnica
TU-AL-01	RF-ALERT-03: sensor não operacional devolve Informative	classifyAlert()	Unidade	Particionamento de Equivalência
TU-AL-02	RF-ALERT-03: sensor KO prevalece sobre temperatura fora	classifyAlert()	Unidade	Particionamento de Equivalência
TU-AL-03	RF-ALERT-03: sensor KO prevalece sobre humidade fora	classifyAlert()	Unidade	Particionamento de Equivalência
TU-AL-04	RF-ALERT-03: sensor KO prevalece sobre ambas variáveis fora	classifyAlert()	Unidade	Particionamento de Equivalência
TU-AL-05	RF-ALERT-03: sensor KO com temperatura extrema	classifyAlert()	Unidade	Particionamento de Equivalência
TU-AL-06	RF-ALERT-01: valores nominais dentro dos limites devolvem Informative	classifyAlert()	Unidade	Particionamento de Equivalência
TU-AL-07	RN-ALERT-02: limite inferior de temperatura é inclusivo	classifyAlert()	Unidade	Análise de Valores Limite
TU-AL-08	RN-ALERT-02: limite superior de temperatura é inclusivo	classifyAlert()	Unidade	Análise de Valores Limite
TU-AL-09	RN-ALERT-02: limite inferior de humidade é inclusivo	classifyAlert()	Unidade	Análise de Valores Limite
TU-AL-10	RN-ALERT-02: limite superior de humidade é inclusivo	classifyAlert()	Unidade	Análise de Valores Limite
TU-AL-11	RN-ALERT-02: ambos os limites inferiores em simultâneo	classifyAlert()	Unidade	Análise de Valores Limite
TU-AL-12	RN-ALERT-02: ambos os limites superiores em simultâneo	classifyAlert()	Unidade	Análise de Valores Limite
TU-AL-13	RF-ALERT-04: uma violação devolve Warning	classifyAlert()	Unidade	Análise de Valores Limite
TU-AL-14	RF-ALERT-04: temperatura muito acima → Warning	classifyAlert()	Unidade	Particionamento de Equivalência
TU-AL-15	RF-ALERT-04: temperatura imediatamente abaixo → Warning	classifyAlert()	Unidade	Análise de Valores Limite
TU-AL-16	RF-ALERT-04: temperatura muito abaixo → Warning	classifyAlert()	Unidade	Particionamento de Equivalência
TU-AL-17	RF-ALERT-04: humidade imediatamente acima → Warning	classifyAlert()	Unidade	Análise de Valores Limite
TU-AL-18	RF-ALERT-04: humidade muito acima → Warning	classifyAlert()	Unidade	Particionamento de Equivalência
TU-AL-19	RF-ALERT-04: humidade imediatamente abaixo → Warning	classifyAlert()	Unidade	Análise de Valores Limite

TU-AL-20	RF-ALERT-04: humidade muito abaixo → Warning	classifyAlert()	Unidade	Particionamento de Equivalência
TU-AL-21	RF-ALERT-05: duas violações devolvem Critical	classifyAlert()	Unidade	Cobertura de Condições Múltiplas
TU-AL-22	RF-ALERT-05: temp e humidade ambas acima → Critical	classifyAlert()	Unidade	Cobertura de Condições Múltiplas
TU-AL-23	RF-ALERT-05: temp e humidade ambas abaixo → Critical	classifyAlert()	Unidade	Cobertura de Condições Múltiplas
TU-AL-24	RF-ALERT-05: temp abaixo + humidade acima → Critical	classifyAlert()	Unidade	Cobertura de Condições Múltiplas
TU-AL-25	RF-ALERT-05: ambas imediatamente acima (fronteira) → Critical	classifyAlert()	Unidade	Cobertura de Condições Múltiplas
TU-AL-26	RF-ALERT-05: ambas imediatamente abaixo (fronteira) → Critical	classifyAlert()	Unidade	Cobertura de Condições Múltiplas
TU-AL-MC01	RN-ALERT-01: tabela de verdade — C1=T C2=T C3=T → Informative	classifyAlert()	Unidade	MC/DC
TU-AL-MC02	RN-ALERT-01: tabela de verdade — C1=T C2=T C3=F → Informative (sensor KO)	classifyAlert()	Unidade	MC/DC
TU-AL-MC03	RN-ALERT-01: tabela de verdade — C1=T C2=F C3=T → Warning (só humidity fora)	classifyAlert()	Unidade	MC/DC
TU-AL-MC04	RN-ALERT-01: tabela de verdade — C1=T C2=F C3=F → Informative (sensor KO)	classifyAlert()	Unidade	MC/DC
TU-AL-MC05	RN-ALERT-01: tabela de verdade — C1=F C2=T C3=T → Warning (só temp fora)	classifyAlert()	Unidade	MC/DC
TU-AL-MC06	RN-ALERT-01: tabela de verdade — C1=F C2=T C3=F → Informative (sensor KO)	classifyAlert()	Unidade	MC/DC
TU-AL-MC07	RN-ALERT-01: tabela de verdade — C1=F C2=F C3=T → Critical (ambas fora)	classifyAlert()	Unidade	MC/DC
TU-AL-MC08	RN-ALERT-01: tabela de verdade — C1=F C2=F C3=F → Informative (sensor KO)	classifyAlert()	Unidade	MC/DC
TU-AL-35	RN-ALERT-04: função usa parâmetro limits, não hard-coded — tightLimits: temp fora apertada	classifyAlert()	Unidade	Particionamento de Equivalência
TU-AL-36	RN-ALERT-04: função usa parâmetro limits, não hard-coded — tightLimits: humidade fora apertada	classifyAlert()	Unidade	Particionamento de Equivalência

TU-AL-37	RN-ALERT-04: função usa parâmetro limits, não hard-coded — wideLimits: temp acima default mas dentro alargado	classifyAlert()	Unidade	Particionamento de Equivalência
TU-AL-38	RN-ALERT-04: função usa parâmetro limits, não hard-coded — wideLimits: humidade acima default mas dentro alargado	classifyAlert()	Unidade	Particionamento de Equivalência
TU-AL-39	RN-ALERT-02: limites inclusivos — fronteira exata minTemp inclusiva	classifyAlert()	Unidade	Análise de Valores Limite
TU-AL-40	RN-ALERT-02: limites inclusivos — fronteira exata maxTemp inclusiva	classifyAlert()	Unidade	Análise de Valores Limite
TU-AL-41	RN-ALERT-02: limites inclusivos — fronteira exata minHumidity inclusiva	classifyAlert()	Unidade	Análise de Valores Limite
TU-AL-42	RN-ALERT-02: limites inclusivos — fronteira exata maxHumidity inclusiva	classifyAlert()	Unidade	Análise de Valores Limite
TU-AL-43	RN-ALERT-02: limites inclusivos — minTemp-1 fora	classifyAlert()	Unidade	Análise de Valores Limite
TU-AL-44	RN-ALERT-02: limites inclusivos — maxTemp+1 fora	classifyAlert()	Unidade	Análise de Valores Limite
TU-AL-45	RN-ALERT-02: limites inclusivos — minHumidity-1 fora	classifyAlert()	Unidade	Análise de Valores Limite
TU-AL-46	RN-ALERT-02: limites inclusivos — maxHumidity+1 fora	classifyAlert()	Unidade	Análise de Valores Limite
TU-AL-47	RN-ALERT-05: comportamento robusto perante valores extremos — temp=0 (extremo): não-Informative	classifyAlert()	Unidade	Robustez
TU-AL-48	RN-ALERT-05: comportamento robusto perante valores extremos — temp negativa não classificada como Informative	classifyAlert()	Unidade	Robustez
TU-AL-49	RN-ALERT-05: comportamento robusto perante valores extremos — humidity=0 (deserto)	classifyAlert()	Unidade	Robustez
TU-AL-50	RN-ALERT-05: comportamento robusto perante valores extremos — humidity=100 (saturação)	classifyAlert()	Unidade	Robustez
TU-AL-51	RN-ALERT-05: comportamento robusto perante valores extremos — temp=maxTemp e humidity=minHumidity em simultâneo (limites inclusivos)	classifyAlert()	Unidade	Robustez
TU-RA-01	RF-ALERT-06: resolver Warning sem justificação → status resolved	resolveAlert()	Unidade	Particionamento de Equivalência

TU-RA-02	RF-ALERT-06: resolver Critical sem justificação	resolveAlert()	Unidade	Particionamento de Equivalência
TU-RA-03	RF-ALERT-06: resolver Informative sem justificação	resolveAlert()	Unidade	Particionamento de Equivalência
TU-RA-04	RF-ALERT-06: justificação não-vazia preservada em resolve	resolveAlert()	Unidade	Particionamento de Equivalência
TU-RA-05	RN-ALERT-07: justification=undefined coerce a null	resolveAlert()	Unidade	Particionamento de Equivalência
TU-RA-06	RN-ALERT-08: justificação obrigatória [10,500] para ignore — limite inferior justificação (10 chars)	resolveAlert()	Unidade	Análise de Valores Limite
TU-RA-07	RN-ALERT-08: justificação obrigatória [10,500] para ignore — limite superior justificação (500 chars)	resolveAlert()	Unidade	Análise de Valores Limite
TU-RA-08	RN-ALERT-08: justificação obrigatória [10,500] para ignore — valor nominal justificação (250 chars)	resolveAlert()	Unidade	Análise de Valores Limite
TU-RA-09	RN-ALERT-08: justificação obrigatória [10,500] para ignore — justificação real válida	resolveAlert()	Unidade	Particionamento de Equivalência
TU-RA-10	RN-ALERT-08: justificação obrigatória [10,500] para ignore — ignore Critical com justificação válida	resolveAlert()	Unidade	Particionamento de Equivalência
TU-RA-10b	RN-ALERT-08: justificação obrigatória [10,500] para ignore — ignore Informative com justificação válida	resolveAlert()	Unidade	Particionamento de Equivalência
TU-RA-11	RN-ALERT-08: justificação fora dos limites → erro — 9 chars (abaixo do mínimo)	resolveAlert()	Unidade	Análise de Valores Limite
TU-RA-12	RN-ALERT-08: justificação fora dos limites → erro — 501 chars (acima do máximo)	resolveAlert()	Unidade	Análise de Valores Limite
TU-RA-13	RN-ALERT-08: justificação fora dos limites → erro — string vazia	resolveAlert()	Unidade	Análise de Valores Limite
TU-RA-14	RN-ALERT-08: justificação fora dos limites → erro — 1 char	resolveAlert()	Unidade	Análise de Valores Limite
TU-RA-15	RN-ALERT-08: justificação fora dos limites → erro — 1000 chars (muito acima)	resolveAlert()	Unidade	Análise de Valores Limite
TU-RA-16	RN-ALERT-08: justificação fora dos limites → erro — 5 chars ('short')	resolveAlert()	Unidade	Análise de Valores Limite

TU-RA-17	RN-ALERT-09: ações inválidas → erro — 'invalid_action' não reconhecida	resolveAlert()	Unidade	Particionamento de Equivalência
TU-RA-18	RN-ALERT-09: ações inválidas → erro — ação vazia	resolveAlert()	Unidade	Particionamento de Equivalência
TU-RA-19	RN-ALERT-09: ações inválidas → erro — RESOLVE em maiúsculas (case-sensitive)	resolveAlert()	Unidade	Particionamento de Equivalência
TU-RA-20	RN-ALERT-09: ações inválidas → erro — IGNORE em maiúsculas	resolveAlert()	Unidade	Particionamento de Equivalência
TU-RA-21	RN-ALERT-09: ações inválidas → erro — 'dismiss' não suportada	resolveAlert()	Unidade	Particionamento de Equivalência
TU-RA-22	RN-ALERT-09: ações inválidas → erro — action=null	resolveAlert()	Unidade	Particionamento de Equivalência
TU-RA-23	RN-ALERT-09: ações inválidas → erro — action=undefined	resolveAlert()	Unidade	Particionamento de Equivalência
TU-RA-24	DEF-01: resolveAlert muta o objeto de entrada — DEF-01: resolve muta alert.status	resolveAlert()	Unidade	Teste de Defeito
TU-RA-25	DEF-01: resolveAlert muta o objeto de entrada — DEF-01: ignore muta alert.status e justification	resolveAlert()	Unidade	Teste de Defeito
TU-RA-25b	DEF-01: resolveAlert muta o objeto de entrada — DEF-01: retorno é a mesma referência do input	resolveAlert()	Unidade	Teste de Defeito
TU-RA-26	RN-ALERT-10: contrato do objeto retornado — resultado de resolve, tem propriedade status	resolveAlert()	Unidade	Contrato
TU-RA-27	RN-ALERT-10: contrato do objeto retornado — resultado de resolve, tem propriedade justification	resolveAlert()	Unidade	Contrato
TU-RA-28	RN-ALERT-10: contrato do objeto retornado — ignore retorna status='ignored'	resolveAlert()	Unidade	Contrato
TU-RA-29	RN-ALERT-10: contrato do objeto retornado — ignore retorna justification não-nula	resolveAlert()	Unidade	Contrato
TU-RA-30	RN-ALERT-10: contrato do objeto retornado — resolve, retorna um objeto (não null/primitivo)	resolveAlert()	Unidade	Contrato
TU-RA-MC01	RN-ALERT-11: pares MC/DC de resolveAlert — C1=T (ignore), justif válida → ignored	resolveAlert()	Unidade	MC/DC

TU-RA-MC02	RN-ALERT-11: pares MC/DC de resolveAlert — C1=F (resolve), justif válida → resolved (C1 muda)	resolveAlert()	Unidade	MC/DC
TU-RA-MC03	RN-ALERT-11: pares MC/DC de resolveAlert — C1=T, justif curta (C2=T) → throw (C2 muda)	resolveAlert()	Unidade	MC/DC
TU-RA-MC04	RN-ALERT-11: pares MC/DC de resolveAlert — C1=T, justif longa (C3=T) → throw (C3 muda)	resolveAlert()	Unidade	MC/DC
TU-A-1.01	RF-AUDIT-01: cada operação de escrita produz uma entrada — Technical Manager regista created_plan	logAction()	Unidade	Particionamento de Equivalência
TU-A-1.02	RF-AUDIT-01: cada operação de escrita produz uma entrada — Admin regista created_user	logAction()	Unidade	Particionamento de Equivalência
TU-A-1.03	RF-AUDIT-01: cada operação de escrita produz uma entrada — Technician regista registered_measurement	logAction()	Unidade	Particionamento de Equivalência
TU-A-1.04	RF-AUDIT-01: cada operação de escrita produz uma entrada — Admin regista imported_herbs	logAction()	Unidade	Particionamento de Equivalência
TU-A-1.05	RF-AUDIT-01: cada operação de escrita produz uma entrada — Technical Manager regista transitioned_batch	logAction()	Unidade	Particionamento de Equivalência
TU-A-1.06	RN-AUDIT-01: ação 'created_plan' é auditável	logAction()	Unidade	Particionamento de Equivalência
TU-A-1.06+01	RN-AUDIT-01: ação 'updated_plan' é auditável	logAction()	Unidade	Particionamento de Equivalência
TU-A-1.06+02	RN-AUDIT-01: ação 'deleted_plan' é auditável	logAction()	Unidade	Particionamento de Equivalência
TU-A-1.06+03	RN-AUDIT-01: ação 'created_batch' é auditável	logAction()	Unidade	Particionamento de Equivalência
TU-A-1.06+04	RN-AUDIT-01: ação 'transitioned_batch' é auditável	logAction()	Unidade	Particionamento de Equivalência
TU-A-1.06+05	RN-AUDIT-01: ação 'divided_batch' é auditável	logAction()	Unidade	Particionamento de Equivalência
TU-A-1.06+06	RN-AUDIT-01: ação 'created_task' é auditável	logAction()	Unidade	Particionamento de Equivalência
TU-A-1.06+07	RN-AUDIT-01: ação 'completed_task' é auditável	logAction()	Unidade	Particionamento de Equivalência
TU-A-1.06+08	RN-AUDIT-01: ação 'registered_measurement' é auditável	logAction()	Unidade	Particionamento de Equivalência

TU-A-1.06+09	RN-AUDIT-01: ação 'resolved_alert' é auditável	logAction()	Unidade	Particionamento de Equivalência
TU-A-1.06+10	RN-AUDIT-01: ação 'ignored_alert' é auditável	logAction()	Unidade	Particionamento de Equivalência
TU-A-1.06+11	RN-AUDIT-01: ação 'switched_mode' é auditável	logAction()	Unidade	Particionamento de Equivalência
TU-A-1.06+12	RN-AUDIT-01: ação 'created_user' é auditável	logAction()	Unidade	Particionamento de Equivalência
TU-A-1.06+13	RN-AUDIT-01: ação 'updated_user' é auditável	logAction()	Unidade	Particionamento de Equivalência
TU-A-1.06+14	RN-AUDIT-01: ação 'deleted_user' é auditável	logAction()	Unidade	Particionamento de Equivalência
TU-A-1.06+15	RN-AUDIT-01: ação 'imported_herbs' é auditável	logAction()	Unidade	Particionamento de Equivalência
TU-A-1.06+16	RN-AUDIT-01: ação 'exported_report' é auditável	logAction()	Unidade	Particionamento de Equivalência
TU-A-1.07	RN-AUDIT-03: user deve ter id e role — user=null	logAction()	Unidade	Particionamento de Equivalência
TU-A-1.08	RN-AUDIT-03: user deve ter id e role — user=undefined	logAction()	Unidade	Particionamento de Equivalência
TU-A-1.09	RN-AUDIT-03: user deve ter id e role — user sem role	logAction()	Unidade	Particionamento de Equivalência
TU-A-1.10	RN-AUDIT-03: user deve ter id e role — user sem id	logAction()	Unidade	Particionamento de Equivalência
TU-A-1.11	RN-AUDIT-03: user deve ter id e role — user com id=null	logAction()	Unidade	Particionamento de Equivalência
TU-A-1.12	RN-AUDIT-03: user deve ter id e role — user com role vazia	logAction()	Unidade	Particionamento de Equivalência
TU-A-1.13	RN-AUDIT-02: action deve ser string não vazia e auditável — string vazia	logAction()	Unidade	Particionamento de Equivalência
TU-A-1.14	RN-AUDIT-02: action deve ser string não vazia e auditável — whitespace-only	logAction()	Unidade	Particionamento de Equivalência
TU-A-1.15	RN-AUDIT-02: action deve ser string não vazia e auditável — action=null	logAction()	Unidade	Particionamento de Equivalência
TU-A-1.16	RN-AUDIT-02: action deve ser string não vazia e auditável — action=undefined	logAction()	Unidade	Particionamento de Equivalência
TU-A-1.17	RN-AUDIT-02: action deve ser string não vazia e auditável — ação não auditável (read)	logAction()	Unidade	Particionamento de Equivalência

TU-A-1.18	RN-AUDIT-02: action deve ser string não vazia e auditável — ação desconhecida	logAction()	Unidade	Particionamento de Equivalência
TU-A-1.19	RF-AUDIT-02: isAuditable verifica pertença ao conjunto — ação auditável	isAuditable()	Unidade	Particionamento de Equivalência
TU-A-1.20	RF-AUDIT-02: isAuditable verifica pertença ao conjunto — ação de leitura	isAuditable()	Unidade	Particionamento de Equivalência
TU-A-1.21	RF-AUDIT-02: isAuditable verifica pertença ao conjunto — string vazia	isAuditable()	Unidade	Particionamento de Equivalência
TU-A-1.22	RF-AUDIT-02: isAuditable verifica pertença ao conjunto — null não throws	isAuditable()	Unidade	Particionamento de Equivalência
TU-A-1.23	RF-AUDIT-02: isAuditable verifica pertença ao conjunto — undefined não throws	isAuditable()	Unidade	Particionamento de Equivalência
TU-A-1.24	RF-AUDIT-03: getAuditLogs filtra entradas — sem filtros devolve todos	getAuditLogs()	Unidade	Particionamento de Equivalência
TU-A-1.25	RF-AUDIT-03: getAuditLogs filtra entradas — filtra por userId	getAuditLogs()	Unidade	Particionamento de Equivalência
TU-A-1.26	RF-AUDIT-03: getAuditLogs filtra entradas — filtra por ação	getAuditLogs()	Unidade	Particionamento de Equivalência
TU-A-1.27	RF-AUDIT-03: getAuditLogs filtra entradas — filtra por userId AND action (interseção)	getAuditLogs()	Unidade	Particionamento de Equivalência
TU-A-1.28	RF-AUDIT-03: getAuditLogs filtra entradas — filtro sem matches	getAuditLogs()	Unidade	Particionamento de Equivalência
TU-A-2.01	RN-AUDIT-04: store cresce monotonicamente — store inicia vazio	auditLogs / logAction()	Unidade	Análise de Valores Limite
TU-A-2.02	RN-AUDIT-04: store cresce monotonicamente — primeira entrada produz size=1	auditLogs / logAction()	Unidade	Análise de Valores Limite
TU-A-2.03	RN-AUDIT-04: store cresce monotonicamente — segunda entrada produz size=2	auditLogs / logAction()	Unidade	Análise de Valores Limite
TU-A-2.04	RN-AUDIT-04: store cresce monotonicamente — 100 entradas consecutivas guardadas	auditLogs / logAction()	Unidade	Análise de Valores Limite
TU-A-2.05	RN-AUDIT-05: id é armazenado como recebido — id=0 (zero — fronteira)	logAction()	Unidade	Análise de Valores Limite

TU-A-2.06	RN-AUDIT-05: id é armazenado como recebido — id=1 (mínimo positivo)	logAction()	Unidade	Análise de Valores Limite
TU-A-2.07	RN-AUDIT-05: id é armazenado como recebido — id=-1 (negativo)	logAction()	Unidade	Análise de Valores Limite
TU-A-2.08	RN-AUDIT-05: id é armazenado como recebido — id=MAX_SAFE_INTEGER	logAction()	Unidade	Análise de Valores Limite
TU-A-2.09	RN-AUDIT-02: limite de comprimento da action — 0 chars (string vazia)	logAction()	Unidade	Análise de Valores Limite
TU-A-2.10	RN-AUDIT-02: limite de comprimento da action — 1 char não auditável	logAction()	Unidade	Análise de Valores Limite
TU-A-2.11	RN-AUDIT-02: limite de comprimento da action — ação auditável mais curta (deleted_plan, 12 chars)	logAction()	Unidade	Análise de Valores Limite
TU-A-2.12	RN-AUDIT-02: limite de comprimento da action — ação auditável mais longa (registered_measurement, 22 chars)	logAction()	Unidade	Análise de Valores Limite
TU-A-2.13	RN-AUDIT-06: meta é deep-copy — meta omitida → {}	logAction()	Unidade	Análise de Valores Limite
TU-A-2.14	RN-AUDIT-06: meta é deep-copy — meta= {} → {}	logAction()	Unidade	Análise de Valores Limite
TU-A-2.15	RN-AUDIT-06: meta é deep-copy — meta com 1 chave	logAction()	Unidade	Análise de Valores Limite
TU-A-2.16	RN-AUDIT-06: meta é deep-copy — meta é deep copy (não partilhada)	logAction()	Unidade	Análise de Valores Limite
TU-A-3.01	RN-AUDIT-07: tabela de verdade logAction — Linha 1: C1=F → 'Valid user' error	logAction()	Unidade	Cobertura de Condições Múltiplas
TU-A-3.02	RN-AUDIT-07: tabela de verdade logAction — Linha 2: C1=T, C2=F → 'non-empty string' error	logAction()	Unidade	Cobertura de Condições Múltiplas
TU-A-3.03	RN-AUDIT-07: tabela de verdade logAction — Linha 3: C1=T, C2=T, C3=F → 'not auditable' error	logAction()	Unidade	Cobertura de Condições Múltiplas
TU-A-3.04	RN-AUDIT-07: tabela de verdade logAction — Linha 4: C1=T, C2=T, C3=T → sucesso	logAction()	Unidade	Cobertura de Condições Múltiplas
TU-A-3.05	RN-AUDIT-07: pares MC/DC logAction — MC/DC C1 low: user=null → throw	logAction()	Unidade	MC/DC

TU-A-3.06	RN-AUDIT-07: pares MC/DC logAction — MC/DC C1 high: user válido → sucesso	logAction()	Unidade	MC/DC
TU-A-3.07	RN-AUDIT-07: pares MC/DC logAction — MC/DC C2 low: action whitespace → throw	logAction()	Unidade	MC/DC
TU-A-3.08	RN-AUDIT-07: pares MC/DC logAction — MC/DC C2 high: action válida → sucesso	logAction()	Unidade	MC/DC
TU-A-3.09	RN-AUDIT-07: pares MC/DC logAction — MC/DC C3 low: action não auditável → throw	logAction()	Unidade	MC/DC
TU-A-3.10	RN-AUDIT-07: pares MC/DC logAction — MC/DC C3 high: action auditável → sucesso	logAction()	Unidade	MC/DC
TU-A-3.11	RF-AUDIT-02: isAuditable = D1 AND D2 — D1=F D2=F (null)	isAuditable()	Unidade	Cobertura de Condições Múltiplas
TU-A-3.12	RF-AUDIT-02: isAuditable = D1 AND D2 — D1=F D2=F (string vazia)	isAuditable()	Unidade	Cobertura de Condições Múltiplas
TU-A-3.13	RF-AUDIT-02: isAuditable = D1 AND D2 — D1=T D2=F (string válida, fora do set)	isAuditable()	Unidade	Cobertura de Condições Múltiplas
TU-A-3.14	RF-AUDIT-02: isAuditable = D1 AND D2 — D1=T D2=T (string válida, no set)	isAuditable()	Unidade	Cobertura de Condições Múltiplas
TU-A-3.15	RF-AUDIT-03: filtro AND de getAuditLogs — F1=F F2=F (sem filtros) → 3 entradas	getAuditLogs()	Unidade	Cobertura de Condições Múltiplas
TU-A-3.16	RF-AUDIT-03: filtro AND de getAuditLogs — F1=T F2=F (userId=1) → 2 entradas	getAuditLogs()	Unidade	Cobertura de Condições Múltiplas
TU-A-3.17	RF-AUDIT-03: filtro AND de getAuditLogs — F1=F F2=T (action=created_plan) → 2 entradas	getAuditLogs()	Unidade	Cobertura de Condições Múltiplas
TU-A-3.18	RF-AUDIT-03: filtro AND de getAuditLogs — F1=T F2=T (interseção) → 1 entrada	getAuditLogs()	Unidade	Cobertura de Condições Múltiplas
TU-A-4.01	RN-AUDIT-08: contrato e imutabilidade — entry contém todos os campos obrigatórios	logAction() / getAuditLogs()	Unidade	Contrato
TU-A-4.02	RN-AUDIT-08: contrato e imutabilidade — timestamp é ISO 8601 válido	logAction() / getAuditLogs()	Unidade	Contrato

TU-A-4.03	RN-AUDIT-08: contrato e imutabilidade — timestamps consecutivos são não-decrescentes	logAction() / getAuditLogs()	Unidade	Contrato
TU-A-4.04	RN-AUDIT-08: contrato e imutabilidade — auditLogs mantém a referência	logAction() / getAuditLogs()	Unidade	Contrato
TU-A-4.05	RN-AUDIT-08: contrato e imutabilidade — logAction falhado não escreve no store	logAction() / getAuditLogs()	Unidade	Contrato
TU-A-4.06	RN-AUDIT-08: contrato e imutabilidade — getAuditLogs devolve cópia, não a referência	logAction() / getAuditLogs()	Unidade	Contrato
TU-A-0.01	RF-AUDIT-01: logAction regista entrada com user e timestamp (suite original)	logAction()	Unidade	Particionamento de Equivalência
TU-B-0.01	RF-BATCH-02: active → completed sem perdas	transitionBatch()	Unidade	Particionamento de Equivalência
TU-B-0.02	RN-BATCH-03: losses>0 sem justif → throw	transitionBatch()	Unidade	Particionamento de Equivalência
TU-B-0.03	RF-BATCH-02: active → compromised com losses e justif	transitionBatch()	Unidade	Particionamento de Equivalência
TU-B-0.04	RN-BATCH-01: papel não autorizado → throw	transitionBatch()	Unidade	Particionamento de Equivalência
TU-B-1.01	RN-BATCH-01: ALLOWED_ROLES = [Technical Manager, Admin] — Technical Manager pode transitar	transitionBatch()	Unidade	Particionamento de Equivalência
TU-B-1.02	RN-BATCH-01: ALLOWED_ROLES = [Technical Manager, Admin] — Admin pode transitar	transitionBatch()	Unidade	Particionamento de Equivalência
TU-B-1.03	RN-BATCH-01: papéis fora de ALLOWED_ROLES rejeitados — Technician não autorizado	transitionBatch()	Unidade	Particionamento de Equivalência
TU-B-1.04	RN-BATCH-01: papéis fora de ALLOWED_ROLES rejeitados — Role desconhecida não autorizada	transitionBatch()	Unidade	Particionamento de Equivalência
TU-B-1.05	RN-BATCH-01: papéis fora de ALLOWED_ROLES rejeitados — Role vazia não autorizada	transitionBatch()	Unidade	Particionamento de Equivalência
TU-B-1.06	RN-BATCH-02: apenas 'active' permite transição — Active pode ser transicionado	transitionBatch()	Unidade	Particionamento de Equivalência

TU-B-1.07	RN-BATCH-02: apenas 'active' permite transição — Completed não pode ser transicionado	transitionBatch()	Unidade	Particionamento de Equivalência
TU-B-1.08	RN-BATCH-02: apenas 'active' permite transição — Compromised não pode ser transicionado	transitionBatch()	Unidade	Particionamento de Equivalência
TU-B-1.09	RN-BATCH-04: TARGET_STATES = [completed, compromised] — alvo 'completed' válido	transitionBatch()	Unidade	Particionamento de Equivalência
TU-B-1.10	RN-BATCH-04: TARGET_STATES = [completed, compromised] — alvo 'compromised' válido com justif	transitionBatch()	Unidade	Particionamento de Equivalência
TU-B-1.11	RN-BATCH-04: TARGET_STATES = [completed, compromised] — alvo inválido 'pending'	transitionBatch()	Unidade	Particionamento de Equivalência
TU-B-1.12	RN-BATCH-04: TARGET_STATES = [completed, compromised] — 'active' não é alvo válido	transitionBatch()	Unidade	Particionamento de Equivalência
TU-B-1.13	RN-BATCH-03: losses>0 exige justif não-vazia — losses=0 e sem justif	transitionBatch()	Unidade	Particionamento de Equivalência
TU-B-1.14	RN-BATCH-03: losses>0 exige justif não-vazia — losses>0 com justif	transitionBatch()	Unidade	Particionamento de Equivalência
TU-B-1.15	RN-BATCH-03: losses>0 exige justif não-vazia — losses>0 sem justif	transitionBatch()	Unidade	Particionamento de Equivalência
TU-B-1.16	RN-BATCH-03: losses>0 exige justif não-vazia — losses>0 com justif=null	transitionBatch()	Unidade	Particionamento de Equivalência
TU-B-1.17	RN-BATCH-05: inputs obrigatórios — batch=null	transitionBatch()	Unidade	Particionamento de Equivalência
TU-B-1.18	RN-BATCH-05: inputs obrigatórios — user=null	transitionBatch()	Unidade	Particionamento de Equivalência
TU-B-1.19	RN-BATCH-05: inputs obrigatórios — batch=undefined	transitionBatch()	Unidade	Particionamento de Equivalência
TU-B-2.01	RN-BATCH-03: limites de losses — losses=-1 (abaixo de zero, tratado como 0)	transitionBatch()	Unidade	Análise de Valores Limite
TU-B-2.02	RN-BATCH-03: limites de losses — losses=0 (mín)	transitionBatch()	Unidade	Análise de Valores Limite
TU-B-2.03	RN-BATCH-03: limites de losses — losses=1 sem justif (mín+1)	transitionBatch()	Unidade	Análise de Valores Limite
TU-B-2.04	RN-BATCH-03: limites de losses — losses=1 com justif	transitionBatch()	Unidade	Análise de Valores Limite

TU-B-2.05	RN-BATCH-03: limites de losses — losses=100 sem justif	transitionBatch()	Unidade	Análise de Valores Limite
TU-B-2.06	RN-BATCH-03: limites de losses — losses=100 com justif	transitionBatch()	Unidade	Análise de Valores Limite
TU-B-2.07	RN-BATCH-03: comprimento da justificação — justif='' (0 chars) com losses>0	transitionBatch()	Unidade	Análise de Valores Limite
TU-B-2.08	RN-BATCH-03: comprimento da justificação — justif só whitespace	transitionBatch()	Unidade	Análise de Valores Limite
TU-B-2.09	RN-BATCH-03: comprimento da justificação — justif=1 char	transitionBatch()	Unidade	Análise de Valores Limite
TU-B-2.10	RN-BATCH-03: comprimento da justificação — justif=500 chars	transitionBatch()	Unidade	Análise de Valores Limite
TU-B-2.11	RN-BATCH-03: comprimento da justificação — justif=501 chars (sem upper cap)	transitionBatch()	Unidade	Análise de Valores Limite
TU-B-3.01	RN-BATCH-06: tabela de verdade reduzida — Linha 1: C1=F → Unauthorized	transitionBatch()	Unidade	Cobertura de Condições Múltiplas
TU-B-3.02	RN-BATCH-06: tabela de verdade reduzida — Linha 2: C1=T C2=F → Invalid transition	transitionBatch()	Unidade	Cobertura de Condições Múltiplas
TU-B-3.03	RN-BATCH-06: tabela de verdade reduzida — Linha 3: C1=T C2=T C3=F → Invalid target state	transitionBatch()	Unidade	Cobertura de Condições Múltiplas
TU-B-3.04	RN-BATCH-06: tabela de verdade reduzida — Linha 4: C1=T C2=T C3=T C4=F → Justification required	transitionBatch()	Unidade	Cobertura de Condições Múltiplas
TU-B-3.05	RN-BATCH-06: tabela de verdade reduzida — Linha 5: todas condições T → sucesso	transitionBatch()	Unidade	Cobertura de Condições Múltiplas
TU-B-3.06	RN-BATCH-06: pares MC/DC — C1 low: Technician → Unauthorized	transitionBatch()	Unidade	MC/DC
TU-B-3.07	RN-BATCH-06: pares MC/DC — C1 high: Technical Manager → sucesso	transitionBatch()	Unidade	MC/DC
TU-B-3.08	RN-BATCH-06: pares MC/DC — C2 low: compromised → Invalid transition	transitionBatch()	Unidade	MC/DC
TU-B-3.09	RN-BATCH-06: pares MC/DC — C2 high: active → sucesso	transitionBatch()	Unidade	MC/DC
TU-B-3.10	RN-BATCH-06: pares MC/DC — C3 low: target='archived' → Invalid target state	transitionBatch()	Unidade	MC/DC

TU-B-3.11	RN-BATCH-06: pares MC/DC — C3 high: target='compromised' → sucesso	transitionBatch()	Unidade	MC/DC
TU-B-3.12	RN-BATCH-06: pares MC/DC — C4 low: losses=3 sem justif → Justification required	transitionBatch()	Unidade	MC/DC
TU-B-3.13	RN-BATCH-06: pares MC/DC — C4 high: losses=3 com justif → sucesso	transitionBatch()	Unidade	MC/DC
TU-B-3.14	RN-BATCH-03: C4 = D1 OR D2 — D1=F D2=F (losses>0, sem justif) → throw	transitionBatch()	Unidade	Cobertura de Condições Múltiplas
TU-B-3.15	RN-BATCH-03: C4 = D1 OR D2 — D1=F D2=T (losses>0, justif presente) → sucesso	transitionBatch()	Unidade	Cobertura de Condições Múltiplas
TU-B-3.16	RN-BATCH-03: C4 = D1 OR D2 — D1=T D2=F (losses=0, sem justif) → sucesso	transitionBatch()	Unidade	Cobertura de Condições Múltiplas
TU-B-3.17	RN-BATCH-03: C4 = D1 OR D2 — D1=T D2=T (losses=0, justif presente) → sucesso	transitionBatch()	Unidade	Cobertura de Condições Múltiplas
TU-B-4.01	RN-BATCH-07: contrato e imutabilidade — objeto retornado contém novo state	transitionBatch()	Unidade	Contrato
TU-B-4.02	RN-BATCH-07: contrato e imutabilidade — preserva losses original	transitionBatch()	Unidade	Contrato
TU-B-4.03	RN-BATCH-07: contrato e imutabilidade — batch original não é mutado	transitionBatch()	Unidade	Contrato
TU-B-4.04	RN-BATCH-07: contrato e imutabilidade — justification é armazenada no retorno	transitionBatch()	Unidade	Contrato
TU-B-4.05	RN-BATCH-07: contrato e imutabilidade — justification vazia armazenada como ''	transitionBatch()	Unidade	Contrato
TU-1.01	RF-USER-01: validateUserRole aceita papel correto — exact string Technician	validateUserRole()	Unidade	Particionamento de Equivalência
TU-1.02	RF-USER-01: validateUserRole aceita papel correto — exact string Technical Manager	validateUserRole()	Unidade	Particionamento de Equivalência
TU-1.03	RF-USER-01: validateUserRole aceita papel correto — exact string Admin	validateUserRole()	Unidade	Particionamento de Equivalência

TU-1.04	RF-USER-01: validateUserRole aceita papel correto — role presente no array	validateUserRole()	Unidade	Particionamento de Equivalência
TU-1.05	RF-USER-01: validateUserRole aceita papel correto — role presente no array (2º elemento)	validateUserRole()	Unidade	Particionamento de Equivalência
TU-1.06	RN-USER-01: validateUserRole rejeita papel errado — Technician vs Technical Manager	validateUserRole()	Unidade	Particionamento de Equivalência
TU-1.07	RN-USER-01: validateUserRole rejeita papel errado — Technician vs array sem Technician	validateUserRole()	Unidade	Particionamento de Equivalência
TU-1.08	RN-USER-01: validateUserRole rejeita papel errado — Admin vs Technician	validateUserRole()	Unidade	Particionamento de Equivalência
TU-1.09	RF-USER-02: getUserPermissions devolve matriz correta — permissões Technician corretas	getUserPermissions()	Unidade	Particionamento de Equivalência
TU-1.10	RF-USER-02: getUserPermissions devolve matriz correta — permissões Technical Manager corretas	getUserPermissions()	Unidade	Particionamento de Equivalência
TU-1.11	RF-USER-02: getUserPermissions devolve matriz correta — permissões Admin corretas	getUserPermissions()	Unidade	Particionamento de Equivalência
TU-1.12	RN-USER-02: roles fora de VALID_ROLES rejeitadas — InvalidRole	getUserPermissions()	Unidade	Particionamento de Equivalência
TU-1.13	RN-USER-02: roles fora de VALID_ROLES rejeitadas — Guest	getUserPermissions()	Unidade	Particionamento de Equivalência
TU-1.14	RN-USER-02: roles fora de VALID_ROLES rejeitadas — null	getUserPermissions()	Unidade	Particionamento de Equivalência
TU-1.15	RN-USER-02: roles fora de VALID_ROLES rejeitadas — undefined	getUserPermissions()	Unidade	Particionamento de Equivalência
TU-1.16	RN-USER-04: Admin tem permissões exclusivas — Admin pode gerir users; TM não	getUserPermissions()	Unidade	Particionamento de Equivalência
TU-1.17	RN-USER-04: Admin tem permissões exclusivas — Admin pode importar herbs; TM não	getUserPermissions()	Unidade	Particionamento de Equivalência
TU-1.18	RN-USER-04: Admin tem permissões exclusivas — Technician não pode ver audit	getUserPermissions()	Unidade	Particionamento de Equivalência
TU-2.01	RN-USER-05: limites de user.id — id=0 (abaixo do mínimo natural)	validateUserRole()	Unidade	Análise de Valores Limite

TU-2.02	RN-USER-05: limites de user.id — id=1 (mínimo válido)	validateUserRole()	Unidade	Análise de Valores Limite
TU-2.03	RN-USER-05: limites de user.id — id=999999	validateUserRole()	Unidade	Análise de Valores Limite
TU-2.04	RN-USER-05: limites de user.id — id=1000000	validateUserRole()	Unidade	Análise de Valores Limite
TU-2.05	RN-USER-05: limites de user.id — id=-1 (negativo)	validateUserRole()	Unidade	Análise de Valores Limite
TU-2.06	RN-USER-05: limites de user.id — id=null com role=undefined	validateUserRole()	Unidade	Análise de Valores Limite
TU-2.07	RN-USER-05: limites de user.id — id=undefined com role=undefined	validateUserRole()	Unidade	Análise de Valores Limite
TU-2.08	RN-USER-03: limites de comprimento de role — role vazia	validateUserRole()	Unidade	Análise de Valores Limite
TU-2.09	RN-USER-03: limites de comprimento de role — role só whitespace	validateUserRole()	Unidade	Análise de Valores Limite
TU-2.10	RN-USER-03: limites de comprimento de role — role 1 char ('A')	validateUserRole()	Unidade	Análise de Valores Limite
TU-2.11	RN-USER-03: limites de comprimento de role — role mínima válida ('Technician', 10 chars)	validateUserRole()	Unidade	Análise de Valores Limite
TU-2.12	RN-USER-03: limites de comprimento de role — role 'Administrator' (não reconhecida)	validateUserRole()	Unidade	Análise de Valores Limite
TU-2.13	RN-USER-03: limites de comprimento de role — role 100 chars 'A'.repeat(100)	validateUserRole()	Unidade	Análise de Valores Limite
TU-2.14	RN-USER-06: limites do array requiredRoles — array vazio	validateUserRole()	Unidade	Análise de Valores Limite
TU-2.15	RN-USER-06: limites do array requiredRoles — array 1 elemento contendo a role	validateUserRole()	Unidade	Análise de Valores Limite
TU-2.16	RN-USER-06: limites do array requiredRoles — 3 roles em array, user tem 1ª	validateUserRole()	Unidade	Análise de Valores Limite
TU-3.01	RN-USER-07: tabela de verdade validateUserRole — Row 1: C1=F C2=F C3=F → guard error	validateUserRole()	Unidade	Cobertura de Condições Múltiplas
TU-3.02	RN-USER-07: tabela de verdade validateUserRole — Row 2: C1=F → guard error	validateUserRole()	Unidade	Cobertura de Condições Múltiplas

TU-3.03	RN-USER-07: tabela de verdade validateUserRole — Row 3: C1=F (undefined) → guard error	validateUserRole()	Unidade	Cobertura de Condições Múltiplas
TU-3.04	RN-USER-07: tabela de verdade validateUserRole — Row 4: C1=F (primitivo) → guard error	validateUserRole()	Unidade	Cobertura de Condições Múltiplas
TU-3.05	RN-USER-07: tabela de verdade validateUserRole — Row 5: C1=T C2=F → guard error (role vazia)	validateUserRole()	Unidade	Cobertura de Condições Múltiplas
TU-3.06	RN-USER-07: tabela de verdade validateUserRole — Row 6: C1=T C2=F (whitespace) → guard error	validateUserRole()	Unidade	Cobertura de Condições Múltiplas
TU-3.07	RN-USER-07: tabela de verdade validateUserRole — Row 7: C1=T C2=T C3=F → Unauthorized	validateUserRole()	Unidade	Cobertura de Condições Múltiplas
TU-3.08	RN-USER-07: tabela de verdade validateUserRole — Row 8: C1=T C2=T C3=T → true	validateUserRole()	Unidade	Cobertura de Condições Múltiplas
TU-3.09	RN-USER-07: pares MC/DC validateUserRole — MC/DC C1: primitivo não-objeto → guard error	validateUserRole()	Unidade	MC/DC
TU-3.10	RN-USER-07: pares MC/DC validateUserRole — MC/DC C2: whitespace role → guard error	validateUserRole()	Unidade	MC/DC
TU-3.11	RN-USER-07: pares MC/DC validateUserRole — MC/DC C3: wrong role → Unauthorized	validateUserRole()	Unidade	MC/DC
TU-3.12	RN-USER-07: pares MC/DC validateUserRole — MC/DC anchor: correct role → true	validateUserRole()	Unidade	MC/DC
TU-3.13	RN-USER-08: aceita string ou array em requiredRoles — D1=F D2=F (string required, wrong role)	validateUserRole()	Unidade	Cobertura de Condições Múltiplas
TU-3.14	RN-USER-08: aceita string ou array em requiredRoles — D1=F D2=T (string required, correct role)	validateUserRole()	Unidade	Cobertura de Condições Múltiplas
TU-3.15	RN-USER-08: aceita string ou array em requiredRoles — D1=T D2=F (array required, role absent)	validateUserRole()	Unidade	Cobertura de Condições Múltiplas
TU-3.16	RN-USER-08: aceita string ou array em requiredRoles — D1=T D2=T (array required, role present)	validateUserRole()	Unidade	Cobertura de Condições Múltiplas
TU-4.01	RF-USER-02 / RN-USER-09: matriz de permissões completa — Technician sem permissões elevadas	getUserPermissions()	Unidade	Particionamento de Equivalência / Contrato

TU-4.02	RF-USER-02 / RN-USER-09: matriz de permissões completa — Technical Manager: operacionais sim, admin não	getUserPermissions()	Unidade	Particionamento de Equivalência / Contrato
TU-4.03	RF-USER-02 / RN-USER-09: matriz de permissões completa — Admin com todas as permissões	getUserPermissions()	Unidade	Particionamento de Equivalência / Contrato
TU-4.04	RF-USER-02 / RN-USER-09: matriz de permissões completa — getUserPermissions devolve cópia (não mutável)	getUserPermissions()	Unidade	Particionamento de Equivalência / Contrato

Sprint 4

4.1 Introdução

O Sprint 4 teve como objetivo a criação de testes de integração para todos os endpoints da API REST GREENHERB, cobrindo o espaço completo de inputs: cabeçalhos HTTP, payloads JSON, métodos HTTP e respostas esperadas.

Os testes foram implementados com Jest + Supertest, ferramenta explicitamente recomendada na secção 7.1 do enunciado para projetos Node.js, funcionando como equivalente programático ao Postman para testes automatizados. A suite cobre 74 casos de teste distribuídos por cinco recursos da API.

4.2 Tecnologias Utilizadas

Tecnologia	***Objetivo***
Node.js	Backend da API REST
Express	Framework HTTP
JWT	Autenticação e autorização
Jest	Framework de testes
Supertest	Testes HTTP de integração (equivalente ao Postman para Node.js)

4.3 Infraestrutura Implementada

Para que os testes de integração pudessem ser executados, foi necessário criar os seguintes artefactos que não existiam no projeto:

Ficheiro	***Descrição***
src/middleware/authMiddleware.js	Verifica o token JWT e define req.user para os routes protegidos
src/controllers/alertController.js	Controller que delega no alertService existente
src/controllers/batchController.js	Controller que delega no batchService existente
src/controllers/auditController.js	Controller que delega no auditService existente
src/controllers/userController.js	Controller com store em memória e VALID_ROLES do userService
src/app.js	Atualizado para registar todos os routes com authMiddleware por router

4.4 Estratégia de Testes

Foram desenvolvidos testes de integração com Jest + Supertest, executados diretamente sobre a API Express com stores em memória, sem dependência de base de dados externa. Esta abordagem permite uma execução rápida e reprodutível, mantendo a stack o mais próxima possível da produção.

Para cada endpoint foram testadas quatro dimensões do espaço de inputs:

1. Headers HTTP — Authorization (ausente, inválido, expirado, perfil errado, perfil correto) e Content-Type.

2. Métodos HTTP — métodos não suportados que devem devolver 404/405.

3. JSON Payload — válido, campos em falta, tipos errados, valores nos limites.

4. Respostas — código HTTP, estrutura JSON e campos obrigatórios.

4.5 Técnicas de Teste Aplicadas

4.5.1 Particionamento de Equivalência

O particionamento de equivalência foi a técnica dominante, sendo aplicada a todos os endpoints para identificar classes válidas e inválidas de inputs. As principais classes testadas foram:

Perfil de utilizador: Technician (403), Technical Manager (201/200), Admin (201/200), sem token (401), token expirado (401).

Tipo de ação em alertas: resolve (justificação opcional), ignore (justificação obrigatória), ação inválida (422).

Estado de lote: active (permite transição), completed/compromised (rejeita transição).

Role de utilizador: Technician, Technical Manager, Admin (válidos); SuperUser (inválido).

4.5.2 Análise de Valores Limite

A análise de valores limite foi aplicada aos parâmetros numéricos e de comprimento:

Parâmetro	***Intervalo***	***Valores Testados***	***Casos***
Temperatura (ºC)	[18, 28]	17, 18, 23, 28, 29	TI-ALERT-07 a 10
Humidade (%)	[40, 80]	39, 40, 60, 80, 81	TI-ALERT-11 a 14
Justif. ignorar (chars)	[10, 500]	9, 10, 250, 500, 501	TI-ALERT-20 a 24
Username (chars)	[1, 30]	30, 31	TI-AUTH-07, TI-AUTH-08
Rate limiting	max 5 falhas	5, 6 tentativas	TI-AUTH-11

4.5.3 Cobertura de Condições Múltiplas (MCC)

A cobertura de condições múltiplas foi aplicada à função transitionBatch, que envolve quatro condições atómicas:

Expressão lógica: PERMITIR se C1 (role permitido) AND C2 (state=active) AND C3 (targetState válido) AND C4 (losses=0 OU justificação presente).

A tabela de verdade seguinte apresenta os seis casos de teste que cobrem as combinações relevantes:

Caso	*C1: Role permitido*	*C2: State=active*	*C3: targetState válido*	*C4: losses=0 ou justif.*	*Resultado*
TI-BATCH-07	✗ (Technician)	✓	✓	✓	403
TI-BATCH-08	✓	✗ (completed)	✓	✓	422
TI-BATCH-09	✓	✓	✗ (suspended)	✓	422
TI-BATCH-10	✓	✓	✓	✗ (losses>0, sem justif.)	422
TI-BATCH-11	✓	✓	✓	✓	200
TI-BATCH-12	✓	✓	✓	✓ (losses=0)	200

4.6 Resumo dos Casos de Teste

A tabela seguinte resume os 74 casos de teste implementados no ficheiro tests/sprint4.integration.test.js, distribuídos por endpoint e técnica:

Endpoint	*Nº Testes*	*PE*	*VL*	*MCC*
/auth/login	11	7	2	0
/batches	13	7	0	6
/alerts	27	13	8	0
/audit	8	8	0	0
/users	15	15	0	0
TOTAL	74	50	10	6

Legenda: PE — Particionamento de Equivalência; VL — Análise de Valores Limite; MCC — Cobertura de Condições Múltiplas.

4.7 Matriz de Rastreabilidade

A matriz é apresentada por recurso da API. Cada linha relaciona o caso de teste com o requisito/regra de negócio, o endpoint testado, a técnica formal aplicada, o resultado esperado e as pré-condições.

4.7.1 /auth — Autenticação

ID	*Requisito / Regra*	*Endpoint*	*Nível*	*Técnica*	*Resultado Esperado*	*Pré-condições*
TI-AUTH-01	RF-AUTH-01: Login válido emite token	POST /auth/login	Integração	PE	200 + token JWT	User admin/123456 em memória
TI-AUTH-02	RF-AUTH-01: Username inexistente	POST /auth/login	Integração	PE	401 com mensagem	—
TI-AUTH-03	RF-AUTH-01: Password incorreta	POST /auth/login	Integração	PE	401	User admin existe
TI-AUTH-04	RN-AUTH-01: Body vazio	POST /auth/login	Integração	PE	400	—
TI-AUTH-05	RN-AUTH-01: Username em falta	POST /auth/login	Integração	PE	400	Body só com password
TI-AUTH-06	RN-AUTH-01: Password em falta	POST /auth/login	Integração	PE	400	Body só com username
TI-AUTH-07	RN-AUTH-02: Username 30 chars (limite máx.)	POST /auth/login	Integração	VL	401 (comprimento válido)	Username 30 chars inexistente
TI-AUTH-08	DEF-02: Username 31 chars — authcontroller não valida compriment o	POST /auth/login	Integração	VL	401 em vez de 400 esperado (bug)	Username 31 chars
TI-AUTH-09	RN-AUTH-03: Content-	POST /auth/login	Integração	PE	400 (body não parseado)	Header text/plain

	Type text/plain					
TI-AUTH-10	RN-AUTH-04: Método HTTP inválido	GET /auth/login	Integração	PE	404	—
TI-AUTH-11	RN-AUTH-05: Rate limiting — 6 tentativas	POST /auth/login	Integração	VL	429 após 5 tentativas	Rate limiter limpo

4.7.2 /batches — Lotes de Cultivo

ID	***Requisito / Regra***	***Endpoint***	***Nível***	***Técnica***	***Resultado Esperado***	***Pré-condições***
TI-BATCH-01	RN-BATCH-01: Auth obrigatória	POST /batches	Integração	PE	401 sem token	—
TI-BATCH-02	RN-BATCH-01: Technician não cria lote	POST /batches	Integração	PE	403	Token Technician
TI-BATCH-03	RN-BATCH-01: Technical Manager cria lote	POST /batches	Integração	PE	201 + state:active	Token Manager
TI-BATCH-04	RN-BATCH-01: Admin cria lote	POST /batches	Integração	PE	201	Token Admin
TI-BATCH-05	RN-BATCH-02: herbId em falta	POST /batches	Integração	PE	400	Token Admin
TI-BATCH-06	RN-BATCH-03: Leitura pública	GET /batches	Integração	PE	200 + array	Token Technician

	(autenticada)					
TI-BATCH-07	RN-BATCH-04: C1=false — Technician tenta transição	PATCH /batches/:id/transition	Integração	MCC	403	Lote activo; token Tech.
TI-BATCH-08	RN-BATCH-04: C2=false — lote já completed	PATCH /batches/:id/transition	Integração	MCC	422	Lote previamente concluído
TI-BATCH-09	RN-BATCH-04: C3=false — targetState inválido	PATCH /batches/:id/transition	Integração	MCC	422	targetState=suspended
TI-BATCH-10	RN-BATCH-04: C4=false — losses>0 sem justif.	PATCH /batches/:id/transition	Integração	MCC	422	Lote activo com losses:10
TI-BATCH-11	RN-BATCH-04: C1+C2+C3+C4 todas verdadeiras	PATCH /batches/:id/transition	Integração	MCC	200 + state:compromised	Lote activo, losses=5, justif. presente
TI-BATCH-12	RN-BATCH-05: Sem perdas, sem justificação	PATCH /batches/:id/transition	Integração	PE	200 + state:completed	Lote activo, losses=0

TI-BATCH-13	RN-AUTH-06: Token expirado	GET /batches	Integração	PE	401	Token gerado com expiresIn:-1s

4.7.3 /alerts — Alertas Ambientais

ID	***Requisito / Regra***	***Endpoint***	***Nível***	***Técnica***	***Resultado Esperado***	***Pré-condições***
TI-ALERT-01	RN-AUTH-01: Auth obrigatória	POST /alerts	Integração	PE	401 sem token	—
TI-ALERT-02	RN-ALERT-01: Sensor inoperacional → Informative	POST /alerts	Integração	PE	201 + Informative	sensorOK:false
TI-ALERT-03	RN-ALERT-02: Ambos dentro dos limites → Informative	POST /alerts	Integração	PE	201 + Informative	temp:23, hum:60
TI-ALERT-04	RN-ALERT-03: Só temp fora → Warning	POST /alerts	Integração	PE	201 + Warning	temp:35, hum:60
TI-ALERT-05	RN-ALERT-04: Ambos fora → Critical	POST /alerts	Integração	PE	201 + Critical	temp:35, hum:90
TI-ALERT-06	RN-ALERT-05: Payload incompleto	POST /alerts	Integração	PE	400	Body parcial (só temp)
TI-ALERT-07	RN-PLAN-01: Temp limite inf. exacto (18 ºC)	POST /alerts	Integração	VL	201 + Informative	temp:18
TI-ALERT-08	RN-PLAN-01: Temp abaixo do limite (17 ºC)	POST /alerts	Integração	VL	201 + Warning	temp:17
TI-ALERT-09	RN-PLAN-01: Temp limite sup.	POST /alerts	Integração	VL	201 + Informative	temp:28

	exacto (28 ºC)					
TI-ALERT-10	RN-PLAN-01: Temp acima do limite (29 ºC)	POST /alerts	Integração	VL	201 + Warning	temp:29
TI-ALERT-11	RN-PLAN-02: Humidade abaixo do limite (39%)	POST /alerts	Integração	VL	201 + Warning	humidity:39
TI-ALERT-12	RN-PLAN-02: Humidade limite inf. exacto (40%)	POST /alerts	Integração	VL	201 + Informative	humidity:40
TI-ALERT-13	RN-PLAN-02: Humidade limite sup. exacto (80%)	POST /alerts	Integração	VL	201 + Informative	humidity:80
TI-ALERT-14	RN-PLAN-02: Humidade acima do limite (81%)	POST /alerts	Integração	VL	201 + Warning	humidity:81
TI-ALERT-15	RN-ALERT-06: Technician não lista alertas	GET /alerts	Integração	PE	403	Token Technician
TI-ALERT-16	RN-ALERT-06: Technical Manager lista alertas	GET /alerts	Integração	PE	200 + array	Token Manager
TI-ALERT-17	RN-ALERT-06: Admin lista alertas	GET /alerts	Integração	PE	200 + array	Token Admin
TI-ALERT-18	RN-ALERT-07: Resolver sem	PATCH /alerts/:id	Integração	PE	200 + status:resolved	Alerta pendente; token Manager

	justificação (válido)					
TI-ALERT-19	RN-ALERT-08: Ignorar sem justificação	PATCH /alerts/:id	Integração	PE	422	Alerta pendente
TI-ALERT-20	RN-ALERT-08: Justif. 9 chars (abaixo limite)	PATCH /alerts/:id	Integração	VL	422	Alerta pendente
TI-ALERT-21	RN-ALERT-08: Justif. 10 chars (limite inf.)	PATCH /alerts/:id	Integração	VL	200 + status:ignored	Alerta pendente
TI-ALERT-22	RN-ALERT-08: Justif. 250 chars (nominal)	PATCH /alerts/:id	Integração	VL	200	Alerta pendente
TI-ALERT-23	RN-ALERT-08: Justif. 500 chars (limite sup.)	PATCH /alerts/:id	Integração	VL	200	Alerta pendente
TI-ALERT-24	RN-ALERT-08: Justif. 501 chars (acima limite)	PATCH /alerts/:id	Integração	VL	422	Alerta pendente
TI-ALERT-25	RN-ALERT-09: Acção inválida (delete)	PATCH /alerts/:id	Integração	PE	422	Alerta pendente
TI-ALERT-26	RN-ALERT-10: Alerta inexistente	PATCH /alerts/9999	Integração	PE	404	—
TI-ALERT-27	RN-ALERT-06: Technician não resolve alertas	PATCH /alerts/:id	Integração	PE	403	Alerta pendente; token Tech.

4.7.4 /audit — Auditoria

ID	***Requisito / Regra***	***Endpoint***	***Nível***	***Técnica***	***Resultado Esperado***	***Pré-condições***

TI-AUDIT-01	RN-AUTH-01: Auth obrigatória	POST /audit	Integração	PE	401 sem token	—
TI-AUDIT-02	RN-AUDIT-01: Acção auditável cria log	POST /audit	Integração	PE	201 + user/action/timestamp	Token Admin; action:created_plan
TI-AUDIT-03	RN-AUDIT-02: Acção não auditável rejeitada	POST /audit	Integração	PE	400	action:get_plans
TI-AUDIT-04	RN-AUDIT-02: Acção vazia rejeitada	POST /audit	Integração	PE	400	action:""
TI-AUDIT-05	RN-AUDIT-03: Technical Manager não acede a logs	GET /audit	Integração	PE	403	Token Manager
TI-AUDIT-06	RN-AUDIT-03: Admin consulta todos os logs	GET /audit	Integração	PE	200 + array	Token Admin

TI-AUDIT-07	RN-AUDIT-04: Logs criados visíveis no GET	POST /audit + GET /audit	Integração	PE	Entrada com meta.batchId:42	Auditoria limpa; token Admin
TI-AUDIT-08	RN-AUDIT-05: Filtro por acção	GET /audit/filter?action=...	Integração	PE	Só entradas com action pedida	2 logs de acções distintas

4.7.5 /users — Gestão de Utilizadores

ID	***Requisito / Regra***	***Endpoint***	***Nível***	***Técnica***	***Resultado Esperado***	***Pré-condições***
TI-USER-01	RN-AUTH-01: Auth obrigatória	POST /users	Integração	PE	401 sem token	—
TI-USER-02	RN-USER-01: Technician não cria utilizadores	POST /users	Integração	PE	403	Token Technician
TI-USER-03	RN-USER-01: Admin cria utilizadores	POST /users	Integração	PE	201 + utilizador	Token Admin; username único
TI-USER-04	RN-USER-01: Technical Manager cria utilizadores	POST /users	Integração	PE	201	Token Manager
TI-USER-05	RN-USER-02: Role inválida	POST /users	Integração	PE	400	role:SuperUser
TI-USER-06	RN-USER-03: Username em falta	POST /users	Integração	PE	400	Body sem username

TI-USER-07	RN-USER-04: Technician não lista utilizadores	GET /users	Integração	PE	403	Token Technician
TI-USER-08	RN-USER-04: Admin lista utilizadores	GET /users	Integração	PE	200 + array	Token Admin
TI-USER-09	RN-USER-05: Manager vê utilizador por ID	GET /users/:id	Integração	PE	200 + utilizador	Token Manager; ID 1 existe
TI-USER-10	RN-USER-05: Utilizador inexistente	GET /users/9999	Integração	PE	404	Token Admin
TI-USER-11	RN-USER-06: Admin actualiza utilizador	PUT /users/:id	Integração	PE	200 + actualizado	Token Admin; ID 3 existe
TI-USER-12	RN-USER-06: Manager não pode actualizar	PUT /users/:id	Integração	PE	403	Token Manager
TI-USER-13	RN-USER-07: Admin elimina utilizador	DELETE /users/:id	Integração	PE	204	Token Admin; utilizador criado antes
TI-USER-14	RN-USER-07: Technician não elimina	DELETE /users/:id	Integração	PE	403	Token Technician
TI-USER-15	RN-USER-08: Método não suportado (PATCH /users)	PATCH /users	Integração	PE	404	Token Admin

4.8 Resultados da Execução

Todos os 74 testes de integração passam com sucesso. O comando de execução é:

npx jest tests/sprint4.integration.test.js

Resultado obtido:

Test Suites: 1 passed, 1 total

Tests: 74 passed, 74 total

Snapshots: 0 total

Time: ~2.5 s

Métrica	***Resultado***
Statement Coverage	— (testes de integração; cobertura de código via jest --coverage)
Nº testes /auth	11
Nº testes /batches	13
Nº testes /alerts	27
Nº testes /audit	8
Nº testes /users	15
Total de testes	74
Tempo de execução	~2.5 s

4.9 Conclusão

O Sprint 4 concluiu o plano de testes da plataforma GREENHERB, acrescentando 74 testes de integração que cobrem o espaço completo de inputs para todos os endpoints da API REST. A abordagem adotada com Jest + Supertest, ferramenta explicitamente recomendada no enunciado para Node.js, permitiu validar headers, payloads, métodos HTTP e respostas de forma automatizada e reprodutível.

Foram aplicadas sistematicamente as técnicas de particionamento de equivalência, análise de valores limite e cobertura de condições múltiplas, garantindo que os casos de teste não foram escolhidos de forma arbitrária. A matriz de rastreabilidade

estabelece a ligação explícita entre cada caso de teste e o requisito ou regra de negócio que exercita.

5. Sprint 5

5.1 Introdução

O Sprint 5 teve como objetivo a criação de testes White-Box para a funcionalidade de criação de planos de cultivo, materializada na função validatePlan() do ficheiro src/services/planService.js. Ao contrário dos sprints anteriores — focados em testes de unidade e integração derivados de requisitos (caixa-preta) —, este sprint aplica técnicas de caixa-branca diretamente sobre o código produtivo, exercitando cada estrutura de controlo identificada por inspeção.

Os testes foram implementados com Jest no ficheiro tests/planWhiteBox.test.js, ferramenta já estabelecida nos sprints anteriores. A suíte cobre 52 casos de teste mapeados às 16

estruturas if/else if e à decisão implícita do retorno final, atingindo 100% de cobertura em todas as métricas reportadas pelo Istanbul (statements, branches, functions, lines).

5.2 Tecnologias Utilizadas

Tecnologia	Objetivo
Node.js	Backend da API REST
Jest	Framework de testes (execução, asserções, test.each)
Istanbul/nyc (via jest --coverage)	Geração de relatórios de cobertura (statements, branches, funcs, lines)
planService.js	Componente sob teste — função validatePlan()

5.3 Estruturas de Controlo Identificadas

A inspeção do código de validatePlan() revelou 16 estruturas if/else if explícitas, mais uma decisão implícita no return final. A função não contém ciclos while nem for; a única iteração presente é a interna ao método Array.prototype.includes() (biblioteca standard), que não é alvo da análise white-box do componente. Esta ausência é explicitamente documentada na matriz de rastreabilidade como Não-Aplicável.

Estrutura	Linha	Expressão	Cond. atómicas	Operador
IF-01	6	!plan.type || !VALID_PLAN_TYPES.includes(plan.type)	2	OR
IF-02	10	minTemperature === undefined || maxTemperature === undefined	2	OR
IF-03	13	minTemp < 18 || minTemp > 28	2	OR
IF-04	17	maxTemp < 18 || maxTemp > 28	2	OR
IF-05	21	minTemp > maxTemp	1	—
IF-06	26	minHumidity === undefined || maxHumidity === undefined	2	OR
IF-07	29	minHum < 40 || minHum > 80	2	OR
IF-08	33	maxHum < 40 || maxHum > 80	2	OR
IF-09	37	minHum > maxHum	1	—
IF-10	42	minLight === undefined || maxLight === undefined	2	OR
IF-11	45	minLight < 5000 || minLight > 25000	2	OR

IF-12	49	maxLight < 5000 || maxLight > 25000	2	OR
IF-13	53	minLight > maxLight	1	—
IF-14	58	durationDays === undefined	1	—
IF-15	60	durationDays < 1 || durationDays > 365 (else if)	2	OR
IF-16	64	type === 'pontual' && authorizedByResponsible !== true	2	AND
D-RET	69	errors.length === 0 (decisão do retorno valid)	1	—

5.4 Estratégia de Testes

Os testes white-box foram desenvolvidos com Jest, executados diretamente sobre a função validatePlan() sem dependência de servidor HTTP, base de dados ou ficheiros externos. Esta abordagem permite execução em fração de segundo e total isolamento da lógica de domínio.

Para cada estrutura de controlo foram exercitados sistematicamente:

- •1 Ambos os ramos (true/false) de cada decisão simples — cobertura de ramos.
- •2 Todas as combinações exequíveis de condições atómicas em decisões compostas — cobertura de condições múltiplas (MCC).
- •3 O efeito independente de cada condição atómica sobre o resultado da decisão — cobertura MC/DC, aplicada de forma completa em IF-16 (única decisão AND da função, com 4 combinações exequíveis).
- •4 Combinação com análise de valores limite (VL) nas decisões que envolvem ranges numéricos (IF-03, IF-04, IF-07, IF-08, IF-11, IF-12, IF-15).

5.5 Técnicas de Teste Aplicadas

5.5.1 Cobertura de Ramos (Branch Coverage)

Para cada estrutura if foram desenhados casos que exercitam ambos os ramos da decisão. As decisões simples — IF-05, IF-09, IF-13, IF-14 e D-RET — são cobertas com exatamente 2 casos cada (ramo verdadeiro e ramo falso).

5.5.2 Cobertura de Condições Múltiplas (MCC)

Para cada decisão composta foram desenhadas as combinações exequíveis das condições atómicas. A tabela seguinte resume o número de combinações testadas por estrutura:

Estrutura	Operador	Combinações teóricas (2^n)	Combinações exequíveis	Justificação
IF-01	OR (2)	4	3	C1=T ⇒ C2=T por subconjunto

				(falsy implica fora da enumeração)
IF-02, IF-06, IF-10	OR (2)	4	4	Todas exequíveis — min e max são variáveis independentes
IF-03, IF-04, IF-07, IF-08, IF-11, IF-12, IF-15	OR (2)	4	3	T/T infeasible — um número não pode ser simultaneamente < a e > b com a < b
IF-16	AND (2)	4	4	Todas exequíveis — cobertura MCC e MC/DC pleno

Tabela de Verdade MCC — IF-16 (validade do plano pontual): type === 'pontual' && authorizedByResponsible !== true

Caso	C1: type === 'pontual'	C2: authorizedByResponsible !== true	Resultado	Caso de teste
1	F	F	F (plano aceite)	TU-WB-44 (regular + autorizado)
2	F	T	F (plano aceite — C1 curto-circuita)	TU-WB-45 (regular + não autorizado)
3	T	F	F (plano aceite)	TU-WB-46 (pontual + autorizado)
4	T	T	T (erro)	TU-WB-47, TU-WB-47b (pontual sem autorização)

5.5.3 Cobertura MC/DC

A cobertura MC/DC (Modified Condition / Decision Coverage) foi aplicada de forma completa à decisão IF-16, escolhida por ser a única decisão lógica composta com AND (todas as 4 combinações exequíveis), o que permite isolar o efeito de cada condição atómica:

- •5 Par {TU-WB-46, TU-WB-47} demonstra que C2 afeta o resultado isolando C1=T (F→T quando C2 muda de F→T).
- •6 Par {TU-WB-45, TU-WB-47} demonstra que C1 afeta o resultado isolando C2=T (F→T quando C1 muda de F→T).
- •7 Conjunto mínimo MC/DC: {TU-WB-45, TU-WB-46, TU-WB-47} — apenas 3 casos para uma decisão de 2 condições, conforme a regra N+1.

5.5.4 Combinação com Análise de Valores Limite

Para as decisões que envolvem ranges numéricos, a cobertura MCC foi combinada com análise de valores limite, garantindo que cada ramo é exercitado precisamente nos pontos críticos do intervalo:

Parâmetro	Intervalo aceite	Valores testados	Casos de teste
Temperatura mínima (ºC)	[18, 28]	17, 23, 29	TU-WB-08, TU-WB-09, TU-WB-10
Temperatura máxima (ºC)	[18, 28]	17, 23, 29	TU-WB-11, TU-WB-12, TU-WB-13
Humidade mínima (%)	[40, 80]	39, 60, 81	TU-WB-20, TU-WB-21, TU-WB-22
Humidade máxima (%)	[40, 80]	39, 60, 81	TU-WB-23, TU-WB-24, TU-WB-25
Luminosidade mínima (lux)	[5000, 25000]	4999, 15000, 25001	TU-WB-32, TU-WB-33, TU-WB-34
Luminosidade máxima (lux)	[5000, 25000]	4999, 15000, 25001	TU-WB-35, TU-WB-36, TU-WB-37
Duração (dias)	[1, 365]	0, 90, 366	TU-WB-41, TU-WB-42, TU-WB-43

5.6 Resumo dos Casos de Teste

A tabela seguinte resume os 52 casos de teste implementados no ficheiro tests/planWhiteBox.test.js, distribuídos por estrutura de controlo e técnica aplicada:

Grupo de estruturas	Nº Testes	Branch	MCC	MCC+VL	MC/DC	Outros

IF-01 (tipo)	4	0	4	0	0	0
IF-02 a IF-05 (temperatura)	12	2	4	6	0	0
IF-06 a IF-09 (humidade)	12	2	4	6	0	0
IF-10 a IF-13 (luminosidade)	12	2	4	6	0	0
IF-14, IF-15 (duração)	4	1	0	3	0	0
IF-16 (autorização)	5	0	2	0	3	0
D-RET (retorno)	3	3	0	0	0	0
TOTAL	52	10	18	21	3	0

Legenda: Branch — cobertura de ramos exclusiva; MCC — cobertura de condições múltiplas; MCC+VL — MCC combinada com valores limite; MC/DC — cobertura MC/DC plena.

5.7 Matriz de Rastreabilidade

A matriz é apresentada por agrupamento de estruturas. Cada linha relaciona o caso de teste com o requisito/regra de negócio, a estrutura de código exercitada, a técnica formal aplicada, o resultado esperado e as pré-condições.

5.7.1 IF-01 — Validação do tipo de plano

ID	Requisito / Regra	Estrutura	Nível	Técnica	Resultado Esperado	Pré-condições
TU-WB-01	RN-PLAN-WB-01: tipo válido	IF-01 (C1=F, C2=F)	Unidade	MCC	Sem 'Tipo de plano inválido'	type:'regular'
TU-WB-02	RN-PLAN-WB-01: tipo fora da enumeração	IF-01 (C1=F, C2=T)	Unidade	MCC	errors contém 'Tipo de plano inválido'	type:'experimental'
TU-WB-03	RN-PLAN-WB-01: tipo omisso	IF-01 (C1=T)	Unidade	MCC	errors contém 'Tipo de plano inválido'	type omisso

TU-WB-03b	RN-PLAN-WB-01: tipo string vazia	IF-01 (C1=T)	Unidade	MCC	errors contém 'Tipo de plano inválido'	type:''

5.7.2 IF-02 a IF-05 — Validação de temperatura

ID	Requisito / Regra	Estrutura	Nível	Técnica	Resultado Esperado	Pré-condições
TU-WB-04	RF-PLAN-03: temperatura presente	IF-02 (C1=F, C2=F)	Unidade	MCC	Sem 'Temperatura obrigatória'	min e max definidos
TU-WB-05	RF-PLAN-03: max omissa	IF-02 (C1=F, C2=T)	Unidade	MCC	errors contém 'Temperatura obrigatória'	maxTemperature omisso
TU-WB-06	RF-PLAN-03: min omissa	IF-02 (C1=T, C2=F)	Unidade	MCC	errors contém 'Temperatura obrigatória'	minTemperature omisso
TU-WB-07	RF-PLAN-03: ambas omissas	IF-02 (C1=T, C2=T)	Unidade	MCC	errors contém 'Temperatura obrigatória'	min e max omissos
TU-WB-08	RN-PLAN-01: minTemp=23 interior	IF-03 (C1=F, C2=F)	Unidade	MCC+VL	Sem erro de minTemp	minTemperature:23
TU-WB-09	RN-PLAN-01: minTemp=17 abaixo	IF-03 (C1=T, C2=F)	Unidade	MCC+VL	errors contém erro de minTemp	minTemperature:17
TU-WB-10	RN-PLAN-01: minTemp=29 acima	IF-03 (C1=F, C2=T)	Unidade	MCC+VL	errors contém erro de minTemp	minTemperature:29

TU-WB-11	RN-PLAN-08: maxTemp=23 interior	IF-04 (C1=F, C2=F)	Unidade	MCC+VL	Sem erro de maxTemp	maxTemperature:23
TU-WB-12	RN-PLAN-08: maxTemp=17 abaixo	IF-04 (C1=T, C2=F)	Unidade	MCC+VL	errors contém erro de maxTemp	maxTemperature:17
TU-WB-13	RN-PLAN-08: maxTemp=29 acima	IF-04 (C1=F, C2=T)	Unidade	MCC+VL	errors contém erro de maxTemp	maxTemperature:29
TU-WB-14	RN-PLAN-07: min ≤ max coerente	IF-05 (F)	Unidade	Branch	Sem erro de coerência	min=18, max=28
TU-WB-15	RN-PLAN-07: min > max incoerente	IF-05 (T)	Unidade	Branch	errors contém erro de coerência	min=28, max=18

5.7.3 IF-06 a IF-09 — Validação de humidade

ID	Requisito / Regra	Estrutura	Nível	Técnica	Resultado Esperado	Pré-condições
TU-WB-16	RF-PLAN-04: humidade presente	IF-06 (C1=F, C2=F)	Unidade	MCC	Sem 'Humidade obrigatória'	min e max definidos
TU-WB-17	RF-PLAN-04: maxHumidity omissa	IF-06 (C1=F, C2=T)	Unidade	MCC	errors contém 'Humidade obrigatória'	maxHumidity omisso
TU-WB-18	RF-PLAN-04: minHumidity omissa	IF-06 (C1=T, C2=F)	Unidade	MCC	errors contém 'Humidade obrigatória'	minHumidity omisso
TU-WB-19	RF-PLAN-04: ambas omissas	IF-06 (C1=T, C2=T)	Unidade	MCC	errors contém 'Humidade obrigatória'	min e max omissos

TU-WB-20	RN-PLAN-02: minHum=60 interior	IF-07 (C1=F, C2=F)	Unidade	MCC+VL	Sem erro de minHum	minHumidity:60
TU-WB-21	RN-PLAN-02: minHum=39 abaixo	IF-07 (C1=T, C2=F)	Unidade	MCC+VL	errors contém erro de minHum	minHumidity:39
TU-WB-22	RN-PLAN-02: minHum=81 acima	IF-07 (C1=F, C2=T)	Unidade	MCC+VL	errors contém erro de minHum	minHumidity:81
TU-WB-23	RN-PLAN-09: maxHum=60 interior	IF-08 (C1=F, C2=F)	Unidade	MCC+VL	Sem erro de maxHum	maxHumidity:60
TU-WB-24	RN-PLAN-09: maxHum=39 abaixo	IF-08 (C1=T, C2=F)	Unidade	MCC+VL	errors contém erro de maxHum	maxHumidity:39
TU-WB-25	RN-PLAN-09: maxHum=81 acima	IF-08 (C1=F, C2=T)	Unidade	MCC+VL	errors contém erro de maxHum	maxHumidity:81
TU-WB-26	RN-PLAN-11: min ≤ max coerente	IF-09 (F)	Unidade	Branch	Sem erro de coerência	min=40, max=80
TU-WB-27	RN-PLAN-11: min > max incoerente	IF-09 (T)	Unidade	Branch	errors contém erro de coerência	min=80, max=40

5.7.4 IF-10 a IF-13 — Validação de luminosidade

ID	Requisito / Regra	Estrutura	Nível	Técnica	Resultado Esperado	Pré-condições
TU-WB-28	RF-PLAN-05: luminosidade presente	IF-10 (C1=F, C2=F)	Unidade	MCC	Sem 'Luminosidade obrigatória'	min e max definidos

TU-WB-29	RF-PLAN-05: maxLight omissa	IF-10 (C1=F, C2=T)	Unidade	MCC	errors contém 'Luminosidade obrigatória'	maxLight omisso
TU-WB-30	RF-PLAN-05: minLight omissa	IF-10 (C1=T, C2=F)	Unidade	MCC	errors contém 'Luminosidade obrigatória'	minLight omisso
TU-WB-31	RF-PLAN-05: ambas omissas	IF-10 (C1=T, C2=T)	Unidade	MCC	errors contém 'Luminosidade obrigatória'	min e max omissos
TU-WB-32	RN-PLAN-03: minLight=15000 interior	IF-11 (C1=F, C2=F)	Unidade	MCC+VL	Sem erro de minLight	minLight:15000
TU-WB-33	RN-PLAN-03: minLight=4999 abaixo	IF-11 (C1=T, C2=F)	Unidade	MCC+VL	errors contém erro de minLight	minLight:4999
TU-WB-34	RN-PLAN-03: minLight=25001 acima	IF-11 (C1=F, C2=T)	Unidade	MCC+VL	errors contém erro de minLight	minLight:25001
TU-WB-35	RN-PLAN-10: maxLight=15000 interior	IF-12 (C1=F, C2=F)	Unidade	MCC+VL	Sem erro de maxLight	maxLight:15000
TU-WB-36	RN-PLAN-10: maxLight=4999 abaixo	IF-12 (C1=T, C2=F)	Unidade	MCC+VL	errors contém erro de maxLight	maxLight:4999
TU-WB-37	RN-PLAN-10: maxLight=25001 acima	IF-12 (C1=F, C2=T)	Unidade	MCC+VL	errors contém erro de maxLight	maxLight:25001
TU-WB-38	RN-PLAN-12: min ≤ max coerente	IF-13 (F)	Unidade	Branch	Sem erro de coerência	min=5000, max=25000
TU-WB-39	RN-PLAN-12: min > max incoerente	IF-13 (T)	Unidade	Branch	errors contém erro de coerência	min=25000, max=5000

5.7.5 IF-14 e IF-15 — Duração do plano (if / else if)

ID	Requisito / Regra	Estrutura	Nível	Técnica	Resultado Esperado	Pré-condições
TU-WB-40	RF-PLAN-06: duração omissa	IF-14 (T) — IF-15 não avaliado	Unidade	Branch + cadeia	errors contém 'Duração obrigatória'	durationDays omisso
TU-WB-41	RN-PLAN-04: duração nominal	IF-14 (F) → IF-15 (C1=F, C2=F)	Unidade	Branch + MCC	Sem erros de duração	durationDays:90
TU-WB-42	RN-PLAN-04: durationDays=0 abaixo	IF-15 (C1=T, C2=F)	Unidade	MCC+VL	errors contém 'Duração fora...'	durationDays:0
TU-WB-43	RN-PLAN-04: durationDays=366 acima	IF-15 (C1=F, C2=T)	Unidade	MCC+VL	errors contém 'Duração fora...'	durationDays:366

5.7.6 IF-16 — Plano pontual exige autorização (decisão AND)

ID	Requisito / Regra	Estrutura	Nível	Técnica	Resultado Esperado	Pré-condições
TU-WB-44	RN-PLAN-05/06: regular + autorizado	IF-16 (C1=F, C2=F)	Unidade	MCC + MC/DC	Plano aceite	type:'regular', authorized:true
TU-WB-45	RN-PLAN-05/06: regular + não autorizado	IF-16 (C1=F, C2=T)	Unidade	MCC + MC/DC	Plano aceite (C1 curto-circuita)	type:'regular', authorized:false
TU-WB-46	RN-PLAN-06: pontual + autorizado	IF-16 (C1=T, C2=F)	Unidade	MCC + MC/DC	valid:true, sem erros	type:'pontual', authorized:true

TU-WB-47	RN-PLAN-05: pontual + não autorizado	IF-16 (C1=T, C2=T)	Unidade	MCC + MC/DC	errors contém 'Plano pontual exige autorização...'	type:'pontual', authorized:false
TU-WB-47b	RN-PLAN-05: pontual + autorização omissa	IF-16 (C1=T, C2=T)	Unidade	MCC	errors contém 'Plano pontual exige autorização...'	type:'pontual', authorized omisso

5.7.7 D-RET — Decisão implícita do return

ID	Requisito / Regra	Estrutura	Nível	Técnica	Resultado Esperado	Pré-condições
TU-WB-48	RF-PLAN-VAL: plano sem erros	D-RET (T)	Unidade	Branch	valid:true, errors.length===0	Plano base válido
TU-WB-49	RF-PLAN-VAL: plano com erros	D-RET (F)	Unidade	Branch	valid:false	type:'experimental'
TU-WB-50	Ausência de early-return	Todos os errors.push	Unidade	Branch + acumulação	valid:false e 8 erros simultâneos em errors	Plano com violação em todos os campos

5.8 Resultados da Execução

Todos os 52 testes white-box passam com sucesso. O comando de execução é:

```
npx jest tests/planWhiteBox.test.js --coverage --
collectCoverageFrom='src/services/planService.js'
```

Resultado obtido:

Métrica	Resultado
Test Suites	1 passed, 1 total
Tests	52 passed, 52 total

Statement Coverage	100%
Branch Coverage	100%
Function Coverage	100%
Line Coverage	100%
Tempo de execução	~0.65 s

5.9 Conclusão

O Sprint 5 completou o plano de testes white-box da plataforma GREENHERB, acrescentando 52 testes de unidade que cobrem na totalidade as 16 estruturas if/else if e a decisão implícita do retorno da função validatePlan(). A análise foi conduzida por inspeção direta do código produtivo, com cada caso de teste explicitamente mapeado à estrutura de controlo, condição atómica e técnica aplicada.

Foram aplicadas sistematicamente as três técnicas de caixa-branca exigidas pelo enunciado: cobertura de ramos, cobertura de condições múltiplas (MCC) e cobertura MC/DC, esta última de forma plena na decisão IF-16 (única decisão AND da função). As decisões compostas que combinam ranges numéricos foram adicionalmente reforçadas com análise de valores limite, gerando 21 casos de teste híbridos MCC+VL.

Combinações logicamente infeasible — designadamente o par (T, F) de IF-01 (mascarado por subconjunto) e o par (T, T) das comparações de range (mutuamente excludentes) — foram explicitamente identificadas e justificadas na matriz, evitando inflação artificial do número de testes. A ausência de ciclos while e for no código produtivo foi também documentada como Não-Aplicável, em vez de omitida.

O resultado final é 100% de cobertura em todas as métricas reportadas pelo Istanbul (statements, branches, functions, lines), com tempo de execução inferior a 1 segundo, demonstrando que a aplicação rigorosa de técnicas formais de caixa-branca permite atingir cobertura total sem recurso a testes redundantes ou exploratórios.

6. Sprint 6 — Duplos de Teste

O Sprint 6 centrou-se na aplicação do padrão de duplos de teste (test doubles) para isolar o MeasurementService das suas dependências externas — hardware de sensores e serviços de notificação — permitindo testar a lógica de negócio de forma determinística, sem rede, hardware real ou serviços de terceiros. Os testes foram implementados com Jest no ficheiro tests/sprint6_doubles_test.js, exercitando o serviço via injeção de dependência.

6.1 Arquitetura e Padrão de Injeção de Dependência

O MeasurementService possui duas dependências externas injetadas no construtor — TemperatureGateway e NotificationGateway — e uma decisão de classificação delegada ao módulo alertService. O fluxo principal processAutomaticReading contém três estruturas de decisão atómicas (condição de sensor, condição de limites de temperatura, condição de limites de humidade) e uma guarda de efeito lateral para o envio de notificações apenas para alertas Warning/Critical.

6.1.1 Justificação dos Duplos

TemperatureGateway → STUB: A medição de temperatura é um processo de leitura passiva. O MeasurementService necessita apenas de controlar os valores devolvidos (temperatura dentro/fora dos limites, sensor avariado), sem verificar se o gateway foi chamado. O stub substitui o hardware real com respostas pré-configuradas, sem overhead de verificação de interações.

NotificationGateway → MOCK: O envio de notificações é um efeito lateral observável e crítico. É necessário garantir que o gateway foi efetivamente chamado, com os argumentos corretos (destinatário, tipo de alerta, canal) e o número esperado de vezes. O mock regista chamadas e argumentos para verificação posterior.

6.2 Suite A — Stub do TemperatureGateway

Objetivo: substituir o hardware de sensores por um stub com valores controlados e verificar a classificação de alertas produzida pelo MeasurementService. Cobre os requisitos RN-02 (classificação de alertas) e RN-07 (saúde do sensor).

6.2.1 Matriz de Rastreabilidade — Suite A

ID	Requisito / Regra	Componente	Nível	Técnica	Resultado Esperado	Pré-condições

TU-SP6-01	RN-02: leitura nominal dentro dos limites → Informative	MeasurementService (stub)	Unidade	PE	alertType = 'Informative'; temperature = 23, humidity = 60	Stub: temp=23, hum=60, sensorOK=true
TU-SP6-02	RN-02 + AVL: limite inferior de temperatura exato (18 °C) aceite	MeasurementService (stub)	Unidade	AVL (limite exato)	alertType = 'Informative'	Stub: temp=18, hum=60, sensorOK=true
TU-SP6-03	RN-02 + AVL: temperatura imediatamente abaixo do limite inferior (17 °C) rejeitada	MeasurementService (stub)	Unidade	AVL (abaixo do limite)	alertType = 'Warning'	Stub: temp=17, hum=60, sensorOK=true
TU-SP6-04	RN-02 + AVL: limite superior de temperatura exato (28 °C) aceite	MeasurementService (stub)	Unidade	AVL (limite exato)	alertType = 'Informative'	Stub: temp=28, hum=60, sensorOK=true
TU-SP6-05	RN-02 + AVL: temperatura imediatamente acima do limite superior (29 °C) rejeitada	MeasurementService (stub)	Unidade	AVL (acima do limite)	alertType = 'Warning'	Stub: temp=29, hum=60, sensorOK=true
TU-SP6-06	RN-02: sensor avariado (sensorOK=false) prevalece sobre qualquer leitura → Informative	MeasurementService (stub)	Unidade	MCC (C1: !sensorOK=T)	alertType = 'Informative'; reading.sensorOK = false	Stub: temp=35, hum=90, sensorOK=false
TU-SP6-07	RN-02: ambos temperatura e humidade fora dos limites → Critical	MeasurementService (stub)	Unidade	MCC (C2=F e C3=F)	alertType = 'Critical'	Stub: temp=35, hum=90, sensorOK=true
TU-SP6-08	RN-02 + AVL: limite inferior de humidade exato (40 %) aceite	MeasurementService (stub)	Unidade	AVL (limite exato)	alertType = 'Informative'	Stub: temp=23, hum=40, sensorOK=true
TU-SP6-09	RN-02 + AVL: humidade imediatamente abaixo do limite inferior (39 %) rejeitada	MeasurementService (stub)	Unidade	AVL (abaixo do limite)	alertType = 'Warning'	Stub: temp=23, hum=39, sensorOK=true
TU-SP6-10	RN-07: checkSensorHealth delega em isSensorHealthy do stub → sensor operacional	MeasurementService (stub)	Unidade	PE (sensor operacional)	retorna true	Stub configurado com healthy=true

6.2.2 Tabela de Verdade MCC — classifyAlert (3 condições atómicas)

A função classifyAlert avalia três condições atómicas: C1 (sensor avariado), C2 (temperatura dentro dos limites) e C3 (humidade dentro dos limites). A condição C1 provoca um curto-circuito — quando o sensor está avariado, o resultado é sempre Informative independentemente das leituras.

Caso	C1: !sensorOK	C2: tempOK	C3: humidityOK	Resultado	Testes
1	T	—	—	Informative (curto-circuito)	TU-SP6-06
2	F	T	T	Informative	TU-SP6-01, 02, 04, 08
3	F	F	T	Warning	TU-SP6-03, 05
4	F	T	F	Warning	TU-SP6-09
5	F	F	F	Critical	TU-SP6-07

6.2.3 Análise de Valores Limite — Temperatura [18 °C, 28 °C]

Valor	Posição AVL	Resultado Esperado	Teste
17 °C	Imediatamente abaixo do mínimo	Warning	TU-SP6-03
18 °C	Mínimo exato	Informative	TU-SP6-02
23 °C	Interior nominal	Informative	TU-SP6-01
28 °C	Máximo exato	Informative	TU-SP6-04
29 °C	Imediatamente acima do máximo	Warning	TU-SP6-05

6.2.4 Análise de Valores Limite — Humidade [40 %, 80 %]

Nota: o limite superior exato de humidade (80 %) foi identificado como GAP-01 — não coberto nos 25 testes originais. O valor 81 % é coberto pelo teste TU-SP6-24 na Suite C.

Valor	Posição AVL	Resultado Esperado	Teste
39 %	Imediatamente abaixo do mínimo	Warning	TU-SP6-09
40 %	Mínimo exato	Informative	TU-SP6-08
60 %	Interior nominal	Informative	TU-SP6-01

80 %	Máximo exato	⚠ GAP-01 — não coberto nos originais	—
81 %	Imediatamente acima do máximo	Warning	TU-SP6-24 (Suite C)

6.3 Suite B — Mock do NotificationGateway

Objetivo: verificar que o gateway de notificação é chamado com os argumentos corretos, o número certo de vezes, e apenas quando o alerta o justifica — garantindo o efeito lateral observável do envio de notificações.

6.3.1 Matriz de Rastreabilidade — Suite B

ID	Requisito / Regra	Componente	Nível	Técnica	Resultado Esperado	Pré-condições
TU-SP6-11	RN-06: alerta Informative não desencadeia notificação	MeasurementService (mock)	Unidade	PE (ausência de efeito lateral)	mock.bulkCallCount = 0	Mock limpo; stub leitura nominal; 2 destinatários
TU-SP6-12	RN-06: alerta Warning desencadeia exatamente uma chamada a sendBulkNotification	MeasurementService (mock)	Unidade	PE (efeito lateral presente)	mock.bulkCallCount = 1	Mock limpo; stub temp=17; 2 destinatários
TU-SP6-13	RN-06: payload da chamada contém alertType = 'Warning'	MeasurementService (mock)	Unidade	Verificação de argumentos	mock.wasNotifiedWithAlertType('Warning') = true	Mock limpo; stub temp=17
TU-SP6-14	RN-06: alerta Critical desencadeia exatamente uma chamada a sendBulkNotification	MeasurementService (mock)	Unidade	PE (efeito lateral presente)	mock.bulkCallCount = 1	Mock limpo; stub temp=35, hum=90
TU-SP6-15	RN-06: payload da chamada contém alertType = 'Critical'	MeasurementService (mock)	Unidade	Verificação de argumentos	mock.wasNotifiedWithAlertType('Critical') = true	Mock limpo; stub temp=35, hum=90
TU-SP6-16	RN-06: batch enviado contém uma entrada por cada destinatário definido	MeasurementService (mock)	Unidade	Verificação de argumentos	batch.length = 2	Mock limpo; stub Critical; 2 destinatários

TU-SP6-17	RN-06: payload contém o email correto do primeiro destinatário	MeasurementService (mock)	Unidade	Verificação de argumentos	batch[0].recipientEmail = 'responsavel@greenherb.pt'	Mock limpo; destinatários com email definido
TU-SP6-18	RN-06: todas as notificações do batch usam canal 'email'	MeasurementService (mock)	Unidade	Verificação de argumentos	n.channel = 'email' para todos	Mock limpo; stub Critical; 2 destinatários
TU-SP6-19	RN-06: lista de destinatários vazia impede chamada ao gateway mesmo com Critical	MeasurementService (mock)	Unidade	PE (guarda de lista vazia)	mock.bulkCallCount = 0; notificationSent = false	Mock limpo; stub Critical; recipients = []
TU-SP6-20	RN-06: sensor avariado → Informative → gateway não é chamado	MeasurementService (mock)	Unidade	MCC (sensorOK=false)	mock.bulkCallCount = 0 mesmo com 2 destinatários	Mock limpo; stub sensorOK=false, temp=35, hum=90

6.3.2 Tabela de Decisão — Guarda de Envio de Notificação

Apenas os alertas Warning e Critical com destinatários definidos desencadeiam uma chamada a sendBulkNotification.

alertType	recipients.length	notificationSent	mock.bulkCallCount	Testes
Informative	> 0	false	0	TU-SP6-11
Informative (sensor avariado)	> 0	false	0	TU-SP6-20
Warning	> 0	true	1	TU-SP6-12, 13
Critical	> 0	true	1	TU-SP6-14, 15
Critical	= 0	false	0	TU-SP6-19

6.4 Suite C — Cenários Combinados (Stub + Mock)

Objetivo: validar o fluxo end-to-end da unidade — leitura do stub → classificação via alertService → decisão de notificação via mock — confirmando que os dois duplos interagem corretamente e que o serviço produz os efeitos laterais esperados.

6.4.1 Matriz de Rastreabilidade — Suite C

ID	Requisito / Regra	Componente	Nível	Técnica	Resultado Esperado	Pré-condições
TU-SP 6-21	RN-02 + RN-06: leitura nominal → Informative → sem notificação → notificationSent=false	MeasurementService (stub+mock)	Unidade	Combinado	alertType='Informative'; notificationSent=false; bulkCallCount=0	Stub nominal; mock limpo; 2 destinatários
TU-SP 6-22	RN-02 + RN-06: temperatura fora dos limites → Warning → notificação enviada → notificationSent=true	MeasurementService (stub+mock)	Unidade	Combinado	alertType='Warning'; notificationSent=true; bulkCallCount=1	Stub temp=29; mock limpo; 2 destinatários
TU-SP 6-23	RN-02 + RN-06: ambos fora dos limites → Critical → notificação com alertType='Critical'	MeasurementService (stub+mock)	Unidade	Combinado	alertType='Critical'; mock.wasNotifiedWithAlertType('Critical') = true	Stub temp=35, hum=15; mock limpo; 2 destinatários
TU-SP 6-24	RN-02 + RN-06 + AVL: humidade 81 % (AVL acima do máximo) → Warning → notificação enviada	MeasurementService (stub+mock)	Unidade	AVL + Combinado	alertType='Warning'; notificationSent=true	Stub temp=23, hum=81; mock limpo; 2 destinatários
TU-SP 6-25	RN-07: mock.reset() isola contagens entre cenários — segunda leitura nominal não incrementa bulkCallCount	MeasurementService (stub+mock)	Unidade	Isolamento de estado	Após reset: bulkCallCount=0 independentemente do cenário anterior	Mock após chamada Critical prévia; reset manual; stub nominal

6.5 Cobertura Bidirecional — Requisito → Casos de Teste

As linhas assinaladas a amarelo identificam os gaps e defeitos documentados durante o sprint.

Requisito / Regra	Casos de Teste
RN-02: classificação Informative (leitura dentro dos limites)	TU-SP6-01, TU-SP6-02, TU-SP6-04, TU-SP6-08, TU-SP6-21
RN-02: classificação Informative (sensor avariado prevalece)	TU-SP6-06, TU-SP6-20
RN-02: classificação Warning (temperatura fora dos limites)	TU-SP6-03, TU-SP6-05, TU-SP6-22
RN-02: classificação Warning (humidade fora dos limites)	TU-SP6-09, TU-SP6-24
RN-02: classificação Critical (ambos fora dos limites)	TU-SP6-07, TU-SP6-23
RN-06: notificação enviada para Warning e Critical	TU-SP6-12, TU-SP6-13, TU-SP6-14, TU-SP6-15, TU-SP6-22, TU-SP6-23, TU-SP6-24
RN-06: notificação não enviada para Informative	TU-SP6-11, TU-SP6-21
RN-06: notificação não enviada sem destinatários	TU-SP6-19
RN-06: argumentos do payload (email, canal, alertType, nº destinatários)	TU-SP6-16, TU-SP6-17, TU-SP6-18
RN-07: saúde do sensor (checkSensorHealth)	TU-SP6-10
AVL: limites de temperatura [18, 28]	TU-SP6-02, TU-SP6-03, TU-SP6-04, TU-SP6-05
AVL: limites de humidade [40, 80]	TU-SP6-08, TU-SP6-09, TU-SP6-24
DEF-01: mutação em resolveAlert	— (documentado; não testado nos originais)
GAP-01: limite superior de humidade exato (80 %)	— (não coberto nos originais)
GAP-02: isSensorHealthy → false	— (não coberto nos originais)

6.6 Defeitos e Gaps Identificados

ID	Descrição	Ficheiro	Gravidade	Estado
DEF-01	resolveAlert() muta diretamente o objeto alert recebido e retorna a mesma referência em vez de criar uma cópia. Qualquer código que guarde a referência original observa o estado alterado. Correção: return {	src/services/alertService.js	Média	Aberto

	...alert, status: '...', justification: ... }.			
GAP-01	O limite superior exato de humidade (80 %) não é testado nos 25 testes originais — apenas os limites de temperatura têm os dois extremos cobertos.	tests/sprint6_doubles_test.js	Baixa (cobertura incompleta)	Documentado
GAP-02	O caminho isSensorHealthy → false (sensor inoperacional em checkSensorHealth) não tem caso de teste dedicado.	tests/sprint6_doubles_test.js	Baixa (cobertura incompleta)	Documentado

6.7 Ficheiros Criados no Sprint 6

Ficheiro	Tipo	Descrição
src/gateways/temperatureGateway.js	Interface	Contrato do gateway de temperatura; em produção comunica com sensor físico via MQTT/HTTP
src/gateways/notificationGateway.js	Interface	Contrato do gateway de notificações; em produção comunica com SendGrid/Twilio
src/services/measurementService.js	Serviço	Orquestrador que recebe os gateways por injeção de dependência no construtor
tests/sprint6_doubles_test.js	Testes	25 testes organizados em 3 suites (A — Stub, B — Mock, C — Combinados)

6.8 Resultados da Execução

Test Suites	Tests	Snapshots	Tempo Total
1 passed, 1 total	25 passed, 25 total	0 total	~0.4 s

6.8.1 Distribuição por Técnica

Técnica	Nº de Casos
Particionamento de Equivalência — PE (classe válida/inválida, efeito lateral presente/ausente)	8
Análise de Valores Limite — AVL (limites exatos e vizinhos imediatos)	9

Cobertura de Condições Múltiplas — MCC (sensorOK, tempOK, humidityOK)	4
Verificação de argumentos do mock (email, canal, alertType, nº destinatários)	3
Cenários combinados Stub + Mock (fluxo end-to-end da unidade)	5
Isolamento de estado entre testes (mock.reset())	1
Total Sprint 6	25

6.8.2 Distribuição Acumulada por Nível de Teste

Nível	Sprint 1	Sprint 2	Sprint 3	Sprint 4	Sprint 5	Sprint 6	Total
Unidade	15	31	~50 est.	0	52	25	~173
Integração	0	0	0	74	0	0	74
Sistema	0	0	0	0	0	0	0

6.9 Conclusão

O Sprint 6 demonstrou a eficácia do padrão de duplos de teste para isolar serviços com dependências externas. A combinação de stub para o gateway de temperatura e mock para o gateway de notificações permitiu testar a totalidade do fluxo do MeasurementService de forma determinística, cobrindo as cinco combinações possíveis da tabela de verdade MCC, os valores de fronteira AVL para temperatura e humidade, e os cenários de efeito lateral críticos para a corretude do sistema.

A execução de 25 testes em ~0.4 segundos, sem dependências de rede ou hardware, confirma a adequação da abordagem para integração em pipelines de CI/CD. Os gaps identificados (GAP-01 e GAP-02) e o defeito DEF-01 ficam documentados para endereçamento em sprints futuros.